

รายงานโครงงาน

เว็บแอปพลิเคชันเกมตอบคำถามแข่งขันความรู้แบบเกมมิฟิเคชัน Battle IQ

Battle IQ: A Gamified Trivia Web Application

โดย

673380057-5 นายรัฐภูมิ เกิดพระจัน

673380053-3 นายพีรพัฒน์ ป้องกันยา

673380030-5 นายก้องภพ โชควิริยะ

673380269-0 นายธนกฤต ละครพล

673380266-6 นายณพวิทย์ วงษ์ประเสริฐ

อาจารย์ที่ปรึกษา :

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353002 Principles of Software Design and

Development

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

(ตุลาคม พ.ศ. 2569)

บทคัดย่อ

รัฐภูมิ เกิดพระจีน, พีรพัฒน์ ป้องกันยา, ก้องภพ โชควิริยะ, ธนภฤต ละครพล และณพวิทย์ วงษ์ประเสริฐ. 2569.
เว็บแอปพลิเคชันเกมตอบคำถามแข่งขันความรู้แบบเกมมิฟิเคชัน Battle IQ. รายงานโครงงานรายวิชา
CP353002 Principles of Software Design and Development สาขาวิชาวิทยาการคอมพิวเตอร์
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น.

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาเว็บแอปพลิเคชันเกมตอบคำถามวัดความรู้ตามหมวดหมู่ที่นำแนวคิดเกมมิฟิเคชันมาใช้ ได้แก่ คะแนนที่ขึ้นกับความเร็วในการตอบ โบนัสจากการชนะต่อเนื่อง (win streak) ระบบเลเวล อันดับผู้เล่น และโหมดห้องแข่งขันที่ผู้เล่นหลายคนตอบคำถามชุดเดียวกันพร้อมกัน โดยมุ่งเน้นการออกแบบซอฟต์แวร์ตามสถาปัตยกรรมแบบชั้น (Layered Architecture) หลักการ SOLID และแบบรูปการออกแบบกลุ่มพฤติกรรม ได้แก่ Strategy สำหรับสูตรคิดคะแนน Observer สำหรับการอัปเดตโปรไฟล์เมื่อจบเกม และ Command สำหรับการส่งคำตอบ

ระบบพัฒนาด้วย Java 17 และ Spring Boot ใช้ Spring Data JPA เชื่อมต่อฐานข้อมูล PostgreSQL บนบริการ Neon ส่วนติดต่อผู้ใช้พัฒนาด้วย Thymeleaf ร่วมกับ JavaScript ที่เรียก REST API จัดทำเอกสาร API ด้วย Swagger ทดสอบด้วย JUnit 5 และ Mockito รวม 54 กรณี และนำระบบขึ้นใช้งานจริงบน Render ด้วย Docker ผลการดำเนินงานพบว่าระบบทำงานได้ครบตามขอบเขต

การแยกส่วนรับผิดชอบด้วยแบบรูปการออกแบบทำให้ทีม 5 คนพัฒนาและทดสอบส่วนของตนได้โดยอิสระ และสามารถเพิ่มสูตรคะแนนหรือพฤติกรรมเมื่อจบเกมได้โดยไม่ต้องแก้ไขโค้ดเดิม

คำสำคัญ: เกมตอบคำถาม, เกมมิฟิเคชัน, Spring Boot, Layered Architecture, SOLID, Design Patterns

ABSTRACT

Rattaphum Kerdprachin, Pheerapat Pongkanya, Kongpob Chokviriya, Thanakit Lakornpol and Noppavit Wongprasert. 2026. Battle IQ: A Gamified Trivia Web Application. Project Report for CP353002 Principles of Software Design and Development, Computer Science, College of Computing, Khon Kaen University.

This project aimed to develop a category-based trivia web application that applies gamification: time-dependent scoring, win-streak bonuses, player levels, a leaderboard, and a multiplayer room mode in which several players answer the same question set simultaneously. The design followed a layered architecture, the SOLID principles, and three behavioral design patterns: Strategy for scoring rules, Observer for profile updates when a game ends, and Command for answer submission.

The system was implemented with Java 17 and Spring Boot, using Spring Data JPA with a PostgreSQL database hosted on Neon. The user interface was built with Thymeleaf and JavaScript calling a REST API documented with Swagger. The system was tested with 54 JUnit 5 and Mockito test cases and deployed to Render using Docker. The result is a fully working system within the defined scope; the separation of concerns provided by the design patterns allowed a team of five to develop and test their parts independently, and new scoring rules or end-of-game behaviors can be added without modifying existing code.

Keywords: trivia game, gamification, Spring Boot, layered architecture, SOLID, design patterns

กิตติกรรมประกาศ

รายงานโครงงานฉบับนี้สำเร็จลุล่วงได้ด้วยความช่วยเหลือจากหลายฝ่าย
คณะผู้จัดทำขอขอบพระคุณอาจารย์ผู้สอนรายวิชา CP353002 Principles of Software Design and
Development ที่ให้ความรู้ด้านการออกแบบซอฟต์แวร์ หลักการ SOLID และแบบรูปการออกแบบ
ตลอดจนให้คำแนะนำในการจัดทำโครงงาน

ขอขอบคุณสมาชิกในกลุ่มทุกคนที่ร่วมกันวิเคราะห์ ออกแบบ พัฒนา และทดสอบระบบ
รวมถึงเพื่อนร่วมชั้นที่ช่วยทดลองใช้งานและให้ข้อเสนอแนะ และขอขอบคุณผู้ให้บริการ Neon และ Render
ที่ให้ใช้บริการฐานข้อมูลและเซิร์ฟเวอร์โดยไม่มีค่าใช้จ่ายสำหรับการศึกษา

คณะผู้จัดทำ

ตุลาคม 2569

สารบัญ

บทที่ 1 บทนำ	1
1. ความเป็นมาและความสำคัญของปัญหา	1
2. วัตถุประสงค์	1
3. เป้าหมายและขอบเขต	1
4. ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 งานวิจัยและทฤษฎีที่เกี่ยวข้อง	3
1. ระบบงานที่เกี่ยวข้อง	3
2. หลักการและทฤษฎีที่เกี่ยวข้อง	3
บทที่ 3 วิธีการดำเนินงาน	6
1. ขั้นตอนการดำเนินงาน	6
2. เครื่องมือที่ใช้	6
3. การแบ่งงานและกระบวนการทำงานเป็นทีม	7
บทที่ 4 การวิเคราะห์ระบบและการพัฒนาโปรแกรม	9
1. การวิเคราะห์ระบบ	9
2. การออกแบบระบบ	13
3. การพัฒนาโปรแกรม	27
4. การทดสอบระบบ	30
บทที่ 5 บทสรุป	32
1. สรุปผลการดำเนินโครงการ	32
2. ข้อจำกัดของระบบ	32
3. ปัญหาอุปสรรคและแนวทางแก้ไข	32
4. ข้อเสนอแนะในการพัฒนาต่อไป	33

เอกสารอ้างอิง	34
ภาคผนวก ก คู่มือการติดตั้งและใช้งาน	36
1. การติดตั้งเพื่อพัฒนา	36
2. การใช้งาน	36
ภาคผนวก ข สถิติการทำงานร่วมกันบน GitHub	37
ประวัติผู้เขียน	38

สารบัญตาราง

ตารางที่ 1	เปรียบเทียบคุณลักษณะของระบบเกมตอบคำถามที่เกี่ยวข้องกับ Battle IQ.....	3
ตารางที่ 2	ระยะเวลาดำเนินงาน.....	6
ตารางที่ 3	เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา	6
ตารางที่ 4	การแบ่งหน้าที่รับผิดชอบของสมาชิก	7
ตารางที่ 5	ความต้องการเชิงหน้าที่ (Functional Requirements)	9
ตารางที่ 6	ความต้องการที่ไม่ใช่เชิงหน้าที่ (Non-functional Requirements)	9
ตารางที่ 7	สรุป use case ของระบบ	10
ตารางที่ 8	พจนานุกรมข้อมูลตาราง users.....	22
ตารางที่ 9	พจนานุกรมข้อมูลตาราง user_profiles.....	22
ตารางที่ 10	พจนานุกรมข้อมูลตาราง categories	22
ตารางที่ 11	พจนานุกรมข้อมูลตาราง questions.....	23
ตารางที่ 12	พจนานุกรมข้อมูลตาราง quiz_sessions.....	23
ตารางที่ 13	พจนานุกรมข้อมูลตาราง quiz_details	23
ตารางที่ 14	พจนานุกรมข้อมูลตาราง rooms.....	24
ตารางที่ 15	พจนานุกรมข้อมูลตาราง room_players.....	24
ตารางที่ 16	รายการ endpoint ของ REST API	27
ตารางที่ 17	สูตรคิดคะแนนและคะแนนเต็มต่อรอบ.....	29
ตารางที่ 18	สรุปกรณีทดสอบอัตโนมัติแยกตามคลาส.....	30
ตารางที่ 19	ตัวอย่างกรณีทดสอบแบบค่าขอบและกรณีผิดปกติของ API.....	31
ตารางที่ 20	ปัญหาที่พบบ่อยระหว่างดำเนินงานและแนวทางแก้ไข	32
ตารางที่ 21	จำนวน commit และ pull request ของสมาชิก.....	37

สารบัญภาพ

ภาพที่ 1	Use Case Diagram ของระบบ Battle IQ	10
ภาพที่ 2	Use Case Description (UC-01 ถึง UC-07).....	11
ภาพที่ 3	Use Case Description (UC-08 ถึง UC-13).....	12
ภาพที่ 4	Domain Model ของระบบ	13
ภาพที่ 5	Component Diagram.....	14
ภาพที่ 6	Deployment Diagram	14
ภาพที่ 7	Class Diagram พร้อมตำแหน่งของแบบรูปการออกแบบ	15
ภาพที่ 8	Sequence Diagram การสมัครสมาชิก.....	16
ภาพที่ 9	Sequence Diagram การตอบคำถาม 1 ข้อ (Command และ Strategy).....	16
ภาพที่ 10	Sequence Diagram การจับเกมและอัปเดตโปรไฟล์ (Observer).....	17
ภาพที่ 11	Sequence Diagram การเริ่มแข่งในห้อง.....	17
ภาพที่ 12	Activity Diagram การเล่นเกม 1 รอบ	18
ภาพที่ 13	Activity Diagram การแข่งในห้อง.....	19
ภาพที่ 14	State Diagram ของ QuizSession	20
ภาพที่ 15	State Diagram ของ Room.....	21
ภาพที่ 16	ER Diagram ของฐานข้อมูล Battle IQ	22
ภาพที่ 17	หน้าแรก	25
ภาพที่ 18	หน้าเล่นเกมหลังตอบคำถาม	25
ภาพที่ 19	หน้าสรุปผลการเล่น.....	26
ภาพที่ 20	หน้าห้องแข่งขันเมื่อจบการแข่งขัน.....	26
ภาพที่ 21	หน้าอันดับผู้เล่น.....	27

บทที่ 1 บทนำ

1. ความเป็นมาและความสำคัญของปัญหา

การเรียนรู้ผ่านเกม (game-based learning)

เป็นแนวทางที่ช่วยให้ผู้เรียนทบทวนความรู้ได้อย่างสนุกและต่อเนื่อง

เกมตอบคำถามแบบจับเวลาที่มีการสะสมคะแนน เลเวล และอันดับ สร้างแรงจูงใจให้ผู้เรียนกลับมาเล่นซ้ำ

แอปพลิเคชันในกลุ่มนี้ที่เป็นที่นิยม เช่น Kahoot! และ Quizizz มีจุดเด่นด้านการแข่งขันแบบเรียลไทม์

แต่ส่วนใหญ่เป็นบริการปิดที่ไม่สามารถศึกษาโครงสร้างภายในหรือปรับกติกาคะแนนได้

รายวิชา CP353002 Principles of Software Design and Development

กำหนดให้นักศึกษาออกแบบและพัฒนาระบบด้วย Spring Boot ตามสถาปัตยกรรมแบบชั้น

ประยุกต์ใช้หลักการ SOLID และแบบรูปการออกแบบ (design patterns) อย่างมีเหตุผล

พร้อมทั้งทำงานเป็นทีมผ่าน Git และนำระบบขึ้นใช้งานจริง คณะผู้จัดทำจึงเลือกพัฒนาเกมตอบคำถาม "Battle IQ" ซึ่งมีตรรกะทางธุรกิจที่เหมาะสมกับการใช้แบบรูปการออกแบบ ได้แก่ สูตรคิดคะแนนที่มีหลายแบบ (Strategy)

การแจ้งเหตุการณ์เมื่อจบเกมไปยังส่วนอื่น (Observer)

และการห่อหุ้มการกระทำที่มีหลายขั้นตอนอย่างการตอบคำถาม (Command)

2. วัตถุประสงค์

2.1 เพื่อพัฒนาเว็บแอปพลิเคชันเกมตอบคำถามตามหมวดหมู่ที่มีระบบคะแนน เลเวล win streak อันดับผู้เล่น และโหมดห้องแข่งขันหลายคน

2.2 เพื่อประยุกต์ใช้สถาปัตยกรรมแบบชั้น หลักการ SOLID และแบบรูปการออกแบบกลุ่มพฤติกรรม (Strategy, Observer, Command) ในระบบจริงและอธิบายเหตุผลในการเลือกใช้ได้

2.3 เพื่อออกแบบฐานข้อมูลเชิงสัมพันธ์ที่มีความสัมพันธ์แบบ One-to-One, One-to-Many และ Many-to-Many พร้อม Foreign Key และ Index ที่เหมาะสม

2.4 เพื่อพัฒนา REST API ที่มีเอกสาร การตรวจสอบข้อมูลเข้า และการจัดการข้อผิดพลาดในรูปแบบมาตรฐาน พร้อมชุดทดสอบอัตโนมัติ

2.5 เพื่อฝึกการทำงานเป็นทีมด้วย Git/GitHub ผ่าน branch, pull request และ code review และนำระบบขึ้นใช้งานจริงบนคลาวด์

3. เป้าหมายและขอบเขต

3.1 ขอบเขตด้านผู้ใช้

- 3.1.1 ผู้เล่น: สมัครสมาชิก เข้าสู่ระบบ แก้ไขโปรไฟล์ เลือกหมวดหมู่ เล่นเกม ดูผล ดูประวัติ
ดูอันดับ สร้างและเข้าร่วมห้องแข่งขัน
- 3.1.2 ผู้ดูแลคำถาม: เพิ่ม แก้ไข ลบหมวดหมู่และคำถาม (ในเวอร์ชันนี้ยังไม่แยกสิทธิ์จากผู้เล่น)
- 3.2 ขอบเขตด้านระบบ
 - 3.2.1 รอบการเล่น 1 รอบมีคำถามสูงสุด 5 ข้อ สุ่มจากหมวดหมู่ที่เลือก แต่ละข้อมีเวลาจำกัด
 - 3.2.2 คะแนนต่อข้อคิดจากความถูกต้อง เวลาที่ใช้ และ win streak ของผู้เล่น
 คะแนนรวมสะสมเป็นเลเวล
 - 3.2.3 ห้องแข่งขันรองรับผู้เล่นสูงสุด 8 คน ทุกคนได้คำถามชุดเดียวกัน
 แสดงอันดับในห้องแบบใกล้เคียงเวลาจริงด้วยการ polling
 - 3.2.4 ฐานข้อมูล PostgreSQL 8 ตาราง REST API 22 endpoint พร้อม Swagger UI
- 3.3 สิ่งที่อยู่นอกขอบเขต: ระบบยืนยันตัวตนด้วย token หรือ session ฝั่งเซิร์ฟเวอร์
- การแยกสิทธิ์ผู้ดูแลระบบ การสื่อสารแบบ WebSocket การกู้คืนรหัสผ่าน และแอปพลิเคชันมือถือ

4. ประโยชน์ที่คาดว่าจะได้รับ

- 4.1 ได้ระบบเกมตอบคำถามที่ใช้งานได้จริงผ่านอินเทอร์เน็ต ใช้ทดสอบความรู้และแข่งขันกับเพื่อนได้
- 4.2 สมาชิกทีมเข้าใจการนำสถาปัตยกรรมแบบชั้น SOLID
และแบบรูปการออกแบบไปใช้กับปัญหาจริง
และเห็นผลของการออกแบบที่ดีต่อการทำงานร่วมกันและการขยายระบบ
- 4.3 ได้ประสบการณ์กระบวนการพัฒนาซอฟต์แวร์ครบวงจร ตั้งแต่วิเคราะห์ ออกแบบ พัฒนา ทดสอบ
จนถึงนำขึ้นใช้งานจริง

บทที่ 2 งานวิจัยและทฤษฎีที่เกี่ยวข้อง

1. ระบบงานที่เกี่ยวข้อง

คณะผู้จัดทำศึกษาแอปพลิเคชันเกมตอบคำถามที่ใช้แพร่หลาย 3 ระบบ เพื่อเปรียบเทียบคุณลักษณะกับ Battle IQ ดังตารางที่ 1

ตารางที่ 1 เปรียบเทียบคุณลักษณะของระบบเกมตอบคำถามที่เกี่ยวข้องกับ Battle IQ

คุณลักษณะ	Kahoot!	Quizizz	QuizUp	Battle IQ
คะแนนขึ้นกับเวลาตอบ	มี	มี	มี	มี (โบนัสสูงสุด +10)
โบนัส streak	มี (ตอบถูกติดกันในรอบ)	มี	ไม่มี	มี (ชนะติดกันข้ามรอบ สูงสุด +50%)
เลเวล / อันดับสะสม	ไม่มี	ไม่มี	มี	มี
ห้องแข่งหลายคน	มี (เรียลไทม์)	มี	มี (1 ต่อ 1)	มี (polling, สูงสุด 8 คน)
ผู้ใช้สร้างคำถามเอง	มี	มี	ไม่มี	มี
เปิดเผยโครงสร้างภายใน	ไม่	ไม่	ไม่	ใช่ (open source)

จะเห็นว่า Battle IQ นำคุณลักษณะหลักของระบบที่นิยมมาใช้ ได้แก่ คะแนนตามเวลา โบนัส streak อันดับ และห้องแข่ง โดยเลือกใช้การ polling แทน WebSocket เพื่อลดความซับซ้อนให้เหมาะกับขอบเขตของโครงการ และจุดที่แตกต่างคือการออกแบบภายในที่เปิดเผยและอธิบายได้ด้วยแบบรูปการออกแบบ

2. หลักการและทฤษฎีที่เกี่ยวข้อง

2.1 Spring Boot และสถาปัตยกรรมแบบชั้น (Layered Architecture)

Spring Boot เป็นเฟรมเวิร์กสำหรับพัฒนาแอปพลิเคชัน Java ที่จัดการการตั้งค่าและการสร้าง bean ให้อัตโนมัติผ่าน Dependency Injection แอปพลิเคชันในโครงการนี้แบ่งเป็นชั้นตามหน้าที่ ได้แก่ Presentation (Controller) รับและตอบ HTTP, Service รวมตรรกะทางธุรกิจและขอบเขต transaction, Repository เข้าถึงข้อมูลผ่าน Spring Data JPA และ Domain เก็บ entity กับ enum โดยแต่ละชั้นเรียกได้เฉพาะชั้นที่อยู่ถัดลงไป ทำให้แก้ไขหรือทดสอบแต่ละชั้นแยกกันได้

2.2 หลักการ SOLID

SOLID เป็นหลักการออกแบบเชิงวัตถุ 5 ข้อ ได้แก่ Single Responsibility (คลาสหนึ่งมีเหตุผลในการเปลี่ยนแปลงเพียงอย่างเดียว) Open/Closed (เปิดให้ขยายแต่ปิดต่อการแก้ไข) Liskov Substitution (คลาสลูกใช้แทนคลาสแม่ได้โดยไม่ทำให้ตรรกะผิด) Interface Segregation (แยก interface ตามการใช้งาน ไม่มี interface ก้อนใหญ่) และ Dependency Inversion (โมดูลระดับสูงขึ้นกับ abstraction ไม่ใช่ implementation) หลักการเหล่านี้เป็นเกณฑ์ที่ใช้ตรวจสอบการออกแบบทุกคลาสในโครงการ

2.3 แบบรูปการออกแบบ (Design Patterns)

แบบรูปการออกแบบคือวิธีแก้ปัญหที่เกิดขึ้นในการออกแบบซอฟต์แวร์ซึ่ง Gamma และคณะ (Gang of Four) รวบรวมไว้ 23 แบบ โครงการนี้เลือกใช้กลุ่มพฤติกรรม (behavioral) 3 แบบ และกลุ่มสร้างวัตถุ 1 แบบ ดังนี้

- 2.3.1 Strategy กำหนดกลุ่มของขั้นตอนวิธีที่ interchangeable ได้ไว้หลัง interface เดียว ทำให้เปลี่ยนขั้นตอนวิธีได้ขณะทำงานและเพิ่มแบบใหม่ได้โดยไม่แก่ผู้เรียก ใช้กับสูตรคิดคะแนน
- 2.3.2 Observer กำหนดความสัมพันธ์หนึ่งต่อหลายระหว่างวัตถุ เมื่อวัตถุหนึ่งเปลี่ยนสถานะ ผู้สังเกตการณ์ทุกตัวจะได้รับแจ้ง ใน Spring ใช้ผ่าน ApplicationEvent และ @EventListener ใช้กับเหตุการณ์จับเกม
- 2.3.3 Command ห่อหุ้มคำขอให้เป็นวัตถุ ทำให้แยกผู้ส่งออกจากผู้ทำงาน ทดสอบคำสั่งแยกได้ และต่อย้อนกลับเป็น undo หรือบันทึกประวัติได้ ใช้กับการส่งคำตอบ
- 2.3.4 Factory รวมตรรกะการสร้างวัตถุไว้ที่เดียว ใช้กับการสุ่มชุดคำถาม

2.4 REST API และ Bean Validation

REST (Representational State Transfer) เป็นรูปแบบการออกแบบ API บน HTTP ที่มองทุกอย่างเป็นทรัพยากร ระบุด้วย URL ใช้ HTTP method สื่อความหมาย (GET อ่าน, POST สร้าง, PUT แก้ไข, DELETE ลบ) และใช้ status code บอกผล ส่วน Bean Validation (Jakarta Validation) ใช้ annotation เช่น @NotBlank, @Size, @Pattern บน DTO เพื่อตรวจสอบข้อมูลเข้าก่อนถึงตรรกะทางธุรกิจ

2.5 Object-Relational Mapping และ PostgreSQL

JPA และ Hibernate แปลงระหว่างวัตถุ Java กับตารางในฐานข้อมูลเชิงสัมพันธ์ โดยใช้ annotation ระบุความสัมพันธ์ (@OneToOne, @ManyToOne, @OneToMany) กลยุทธ์การโหลด (fetch type) และการส่งต่อการเปลี่ยนแปลง (cascade) ฐานข้อมูลที่ใช้คือ PostgreSQL ซึ่งรองรับ constraint, index และ transaction ครบถ้วน โครงการใช้บริการ Neon ซึ่งเป็น PostgreSQL แบบ serverless บนคลาวด์

2.6 การเข้ารหัสผ่านด้วย BCrypt

BCrypt เป็นฟังก์ชันแฮชแบบปรับความซ้ำได้ (adaptive) ที่ออกแบบมาสำหรับเก็บรหัสผ่าน มีการเติม salt อัตโนมัติ ทำให้รหัสผ่านเดียวกันได้ค่าแฮชต่างกัน และรับข้อมูลเข้าได้ไม่เกิน 72 ไบต์ ระบบจึงจำกัดความยาวรหัสผ่านไว้ไม่เกิน 72 ตัวอักษร

2.7 การทดสอบซอฟต์แวร์

การทดสอบแบ่งเป็นระดับหน่วย (unit test) ที่ทดสอบคลาสเดียวโดยจำลอง dependency ด้วย Mockito ระดับบูรณาการ (integration test) ที่ทดสอบผ่าน HTTP จริงด้วย MockMvc และระดับระบบที่ทดสอบด้วยมือผ่านหน้าเว็บ การออกแบบกรณีทดสอบใช้เทคนิค equivalence partitioning, boundary value analysis และ negative testing

2.8 Docker และการนำขึ้นใช้งานบนคลาวด์

Docker บรรจุแอปพลิเคชันพร้อมสภาพแวดล้อมที่ต้องการไว้ใน image เดียว ทำให้รันได้เหมือนกันทุกเครื่อง โครงการใช้ multi-stage build สร้าง jar ด้วย JDK แล้วรันด้วย JRE เพื่อให้ image เล็ก และนำขึ้นบริการ Render ซึ่ง build image จาก Dockerfile ใน repository และเปิดให้เข้าถึงผ่าน HTTPS

บทที่ 3 วิธีการดำเนินงาน

1. ขั้นตอนการดำเนินงาน

การดำเนินงานแบ่งเป็น 7 ขั้นตอน โดยขั้นตอนที่ 4 ถึง 6 ทำวนซ้ำเป็นรอบสั้น ๆ ผ่าน pull request

- 1.1 ศึกษาข้อกำหนดของรายวิชา ระบบที่เกี่ยวข้อง และทฤษฎีที่ต้องใช้
- 1.2 วิเคราะห์ความต้องการ กำหนด use case กติกาเกม และ domain model
- 1.3 ออกแบบสถาปัตยกรรม คลาส ฐานข้อมูล และ API
พร้อมเลือกแบบรูปการออกแบบที่เหมาะสมกับแต่ละปัญหา
- 1.4 พัฒนาระบบ โดยแบ่งงานตามส่วนที่แต่ละคนรับผิดชอบ ทำในกิจของตนเองและรวมผ่าน pull request
- 1.5 ทดสอบระดับหน่วย ระดับบูรณาการ และทดสอบระบบด้วยมือ แก้ไขข้อบกพร่องที่พบ
- 1.6 นำระบบขึ้นใช้งานจริงบน Render และทดสอบข้ามบนสภาพแวดล้อมจริง
- 1.7 จัดทำเอกสาร แผนภาพ และสไลด์นำเสนอ

ตารางที่ 2 ระยะเวลาดำเนินงาน

ขั้นตอน	ช่วงเวลา	ผลลัพธ์
ศึกษาและตั้งโปรเจกต์	10 – 20 ก.ย. 2569	repository, โครง Spring Boot, README ฉบับแรก
วิเคราะห์และออกแบบ	20 – 23 ก.ย. 2569	entity 6 ตัว, repository, DTO, domain model
พัฒนารอบที่ 1	23 ก.ย. – 3 ต.ค. 2569	เริ่มเกม จบเกม exception handler validation
พัฒนารอบที่ 2	3 – 7 ต.ค. 2569	ผู้ใช้ คะแนน ส่งคำตอบ CRUD หน้าเว็บ Swagger Docker
พัฒนารอบที่ 3 และทดสอบ	7 – 8 ต.ค. 2569	ห้องแข่งขัน refactor เป็น interface + mapper เทส 54 กรณี
นำขึ้นใช้งานและเอกสาร	8 ต.ค. 2569	URL จริง แผนภาพ 8 ชนิด รายงาน

2. เครื่องมือที่ใช้

ตารางที่ 3 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

ด้าน	เครื่องมือ	การใช้งาน
ภาษาและเฟรมเวิร์ก	Java 17, Spring Boot 4.1 (Web MVC, Data JPA, Validation)	พัฒนา backend และ REST API
ฐานข้อมูล	PostgreSQL บน Neon, Hibernate	เก็บข้อมูล 8 ตาราง
ส่วนติดต่อผู้ใช้	Thymeleaf, HTML, CSS, JavaScript (fetch)	หน้าเว็บ 13 หน้า
ความปลอดภัย	Spring Security Crypto (BCrypt)	เข้ารหัสรหัสผ่าน
เอกสาร API	springdoc-openapi (Swagger UI)	เอกสารและทดลองเรียก API
ทดสอบ	JUnit 5, Mockito, Spring Boot Test, MockMvc	unit และ integration test
Build และ deploy	Maven Wrapper, Docker, Render	build jar, image และนำขึ้นคลาวด์
ควบคุมเวอร์ชัน	Git, GitHub (branch, pull request, review)	ทำงานร่วมกัน 5 คน
แผนภาพ	PlantUML, Mermaid, draw.io	UML และ ER diagram
เครื่องมือพัฒนา	Visual Studio Code, Postman/Swagger	เขียนโค้ดและทดสอบ API

3. การแบ่งงานและกระบวนการทำงานเป็นทีม

ทีมแบ่งงานตามส่วนของระบบเพื่อให้แต่ละคนรับผิดชอบทั้ง backend หน้าเว็บ และการทดสอบของส่วนนั้น ลดการแก้ไฟล์เดียวกันพร้อมกัน ดังตารางที่ 4

ตารางที่ 4 การแบ่งหน้าที่รับผิดชอบของสมาชิก

รหัสนักศึกษา	ชื่อ-นามสกุล	Branch	หน้าที่รับผิดชอบ
673380057-5	นายรัฐภูมิ เกิดพระจัน	rattaphum_6733800575_01	ระบบเล่นเกม: Entity, QuizSessionService, QuestionFactory, GlobalExceptionHandler, หน้าเล่นเกม, Docker
673380053-3	นายพีรพัฒน์ ป้องกันยา	pheerapat_6733800533_01	ผู้ใช้และโปรไฟล์ (Observer), ห้องแข่งขัน, ฐานข้อมูลและ deploy, แผนทดสอบ, เอกสาร
673380030-5	นายก้องภพ โชควิริยะ	kongpob_6733800305_01	คำถามและการส่งคำตอบ (Command), Swagger, โครงหน้าเว็บ

รหัสนักศึกษา	ชื่อ-นามสกุล	Branch	หน้าที่รับผิดชอบ
673380269-0	นายธนกฤต ละครพล	thanakit_6733802690_01	การคิดคะแนน (Strategy), ผลการเรียนและประวัติ, Repository, schema.sql
673380266-6	นายณพวิทย์ วงศ์ประเสริฐ	noppavit_6733802666_01	หมวดหมู่และ Leaderboard, Controller ชุดแรก, Deploy บน Render

กระบวนการ Git ใช้กิ่ง main สำหรับเวอร์ชันส่งมอบ กิ่ง develop สำหรับรวมงาน

และกิ่งส่วนตัวรูปแบบ ชื่อ_รหัส_section สำหรับแต่ละคน ทุกการเปลี่ยนแปลงรวมเข้า develop ผ่าน pull request ที่มีสมาชิกอีกคน review ข้อความ commit ใช้รูปแบบ type: สิ่งที่ทำ เช่น feat: add login endpoint และแต่ละคน commit ด้วยบัญชีของตนเอง

บทที่ 4 การวิเคราะห์ระบบและการพัฒนาโปรแกรม

1. การวิเคราะห์ระบบ

1.1 รายการความต้องการ

ตารางที่ 5 ความต้องการเชิงหน้าที่ (Functional Requirements)

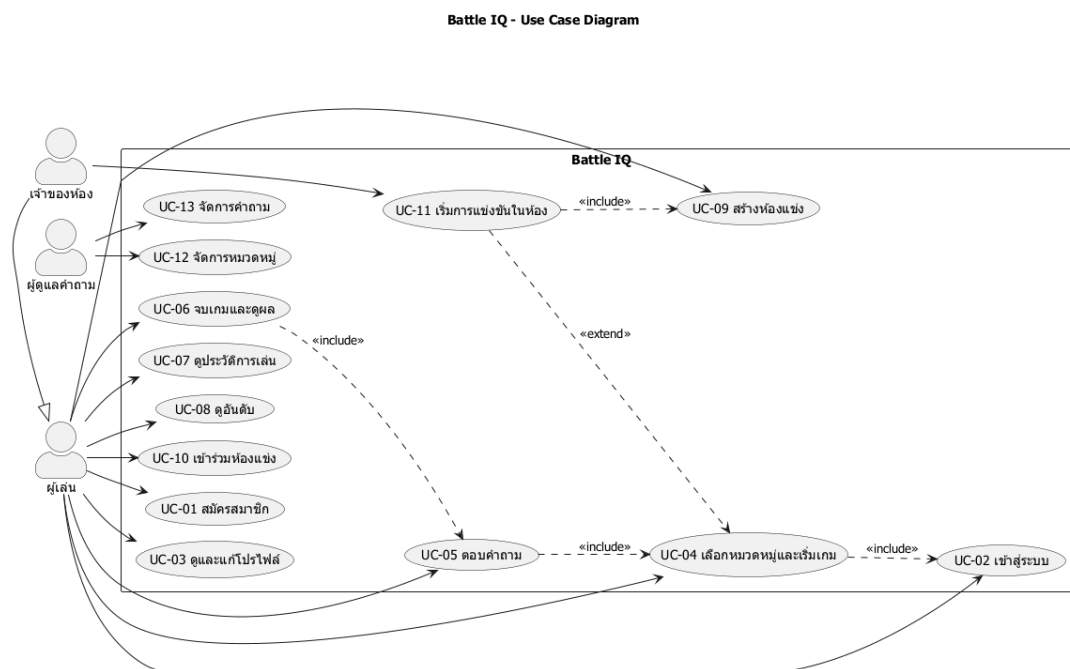
รหัส	ความต้องการ
FR-1	ผู้สมัครสมาชิกด้วยชื่อผู้ใช้ อีเมล รหัสผ่าน และเข้าสู่ระบบได้ รหัสผ่านต้องถูกเข้ารหัส
FR-2	ผู้ใช้เลือกหมวดหมู่และเริ่มรอบการเล่นที่สุ่มคำถามได้สูงสุด 5 ข้อ
FR-3	ระบบตรวจคำตอบ คิดคะแนนตามความถูกต้อง เวลา และ streak และกั้นการตอบซ้ำ
FR-4	เมื่อจบรอบ ระบบรวมคะแนนฝั่งเซิร์ฟเวอร์และอัปเดตคะแนนสะสม เลเวล streak ของผู้เล่น
FR-5	ผู้ดูแลสรุปผลพร้อมเฉลย ประวัติการเล่น และอันดับผู้เล่นได้
FR-6	ผู้สร้างห้องแข่ง เข้าร่วมด้วยรหัส และเจ้าของห้องเริ่มการแข่งขันให้ทุกคนได้คำถามชุดเดียวกัน
FR-7	ผู้ดูแลเพิ่ม แก้ไข ลบหมวดหมู่และคำถามได้ และดูคำถามแบบแบ่งหน้า
FR-8	ระบบมีเอกสาร API (Swagger UI) ที่ทดลองเรียกได้

ตารางที่ 6 ความต้องการที่ไม่ใช่เชิงหน้าที่ (Non-functional Requirements)

รหัส	ความต้องการ
NFR-1	ทุกข้อผิดพลาดของ API ตอบในรูปแบบเดียวกัน (status, message, timestamp) ด้วย status code ที่ถูกต้อง ไม่หลุดเป็น 500 จากข้อมูลเข้าผิดปกติ
NFR-2	ตรรกะทางธุรกิจทุกส่วนมี unit test และ API หลักมี integration test
NFR-3	โค้ดแบ่งชั้นชัดเจน Controller ไม่เรียก Repository โดยตรง Service ขึ้นกับ interface
NFR-4	ระบบเข้าถึงได้ผ่าน URL สาธารณะ และสร้างตารางกับข้อมูลตั้งต้นเองเมื่อติดตั้งครั้งแรก
NFR-5	หน้าเว็บใช้งานได้บนจอคอมพิวเตอร์และจอแคบ

1.2 Use Case Diagram และ Use Case Description

ระบบมี actor 3 บทบาท ได้แก่ ผู้เล่น เจ้าของห้อง (ผู้เล่นที่สร้างห้อง) และผู้ดูแลคำถาม รวม 13 use case ดังภาพที่ 1 และสรุปในตารางที่ 7 รายละเอียดของแต่ละ use case แสดงในภาพที่ 2 และ 3



ภาพที่ 1 Use Case Diagram ของระบบ Battle IQ

ตารางที่ 7 สรุป use case ของระบบ

รหัส	Use case	Actor	สาระสำคัญ
UC-01	สมัครสมาชิก	ผู้เล่น	สร้างบัญชีและโปรไฟล์เริ่มต้น
UC-02	เข้าสู่ระบบ	ผู้เล่น	ตรวจรหัสผ่านและคืนข้อมูลผู้ใช้
UC-03	ดูและแก้ไขโปรไฟล์	ผู้เล่น	ดูคะแนน เลเวล streak แก้ชื่อและรูป
UC-04	เลือกหมวดหมู่และเริ่มเกม	ผู้เล่น	สุ่มคำถาม 5 ข้อ สร้างรอบการเล่น
UC-05	ตอบคำถาม	ผู้เล่น	ตรวจคำตอบ คิดคะแนน บันทึกรายชื่อ
UC-06	จบเกมและดูผล	ผู้เล่น	รวมคะแนน อัปเดตโปรไฟล์ แสดงเฉลย
UC-07	ดูประวัติการเล่น	ผู้เล่น	รายการรอบที่เคยเล่น
UC-08	ดูอันดับ	ผู้เล่น	ผู้เล่นเรียงตามคะแนนรวม
UC-09	สร้างห้องแข่ง	ผู้เล่น	ได้รหัสห้อง 6 ตัว
UC-10	เข้าร่วมห้องแข่ง	ผู้เล่น	เข้าห้องด้วยรหัส
UC-11	เริ่มการแข่งขันในห้อง	เจ้าของห้อง	ทุกคนได้คำถามชุดเดียวกัน
UC-12	จัดการหมวดหมู่	ผู้ดูแลคำถาม	เพิ่ม แก้ไข ลบหมวดหมู่
UC-13	จัดการคำถาม	ผู้ดูแลคำถาม	เพิ่ม แก้ไข ลบคำถาม แบ่งหน้า

Battle IQ - Use Case Description (1/2)

คำอธิบาย use case ประกอบ Use Case Diagram (use-case.png)

UC-01 สมัครสมาชิก	
Actor	ผู้เล่น
Precondition	ยังไม่ล็อกอิน
Main flow	1. เปิดหน้า /register 2. กรอกข้อมูลชื่อ นามสกุล รหัสผ่าน ยืนยันรหัสผ่าน ชื่อที่แสดง 3. ระบบตรวจสอบ (ชื่อ 3-50 ตัว a-z 0-9 _ , รหัสผ่าน 6-72 ตัว) 4. ระบบตรวจสอบและอีเมลซ้ำ 5. เข้าสู่ระบบผ่านด้วย Bcrypt บันทึก users และสร้าง user_profiles (level 1 คะแนน 0) 6. พาไปหน้าเลือกหมวดหมู่
Alternative	3a. ข้อมูลไม่ผ่าน → 400 แสดงข้อความโต้ตอบ / 4a. ชื่อหรืออีเมลซ้ำ → 409
Postcondition	ล็อกอินและโปรไฟล์ สถานะ login ถูกเก็บในเบราว์เซอร์
API	POST /api/v1/users/register

UC-02 เข้าสู่ระบบ	
Actor	ผู้เล่น
Precondition	ล็อกอิน
Main flow	1. กรอกข้อมูลชื่อและรหัสผ่าน 2. ระบบตรวจสอบรหัสผ่านกับค่า Bcrypt 3. คืนข้อมูลผู้ใช้และโปรไฟล์
Alternative	2a. ไม่พบผู้ใช้หรือรหัสผิด → 401 ข้อความเดียวกัน ไม่บอกว่ารหัสผิดหรือไม่
API	POST /api/v1/users/login

UC-03 ดูและแก้ไขโปรไฟล์	
Actor	ผู้เล่น
Precondition	ล็อกอินแล้ว
Main flow	1. เปิด /profile เห็นคะแนนรวม เลเวล streak 2. แก้ชื่อที่แสดงหรือสิ่งกรุป 3. บันทึก
Alternative	2a. ชื่อเกิน 100 หรือสิ่งกรุปเกิน 255 → 400
API	GET / PUT /api/v1/profiles/user/{userId}

UC-04 เลือกหมวดหมู่และเริ่มเกม	
Actor	ผู้เล่น
Precondition	ล็อกอินแล้ว หมวดหมู่มีคำถามอย่างน้อย 1 ข้อ
Main flow	1. เปิด /categories 2. กดเริ่มเล่นที่หมวดหมู่ 3. ระบบสุ่มคำถามสูงสุด 5 ข้อผ่าน QuestionFactory 4. สร้าง quiz_sessions สถานะ IN_PROGRESS และ quiz_details ข้อละ 1 ข้อ 5. พาไปหน้าเล่นเกม
Alternative	3a. หมวดไม่มีคำถาม → 404
API	POST /api/v1/quiz-sessions/start, GET /api/v1/quiz-sessions/{id}/questions

UC-05 ตอบคำถาม	
Actor	ผู้เล่น
Precondition	มีรอบที่กำลังเล่น
Main flow	1. หน้าเล่นเกมแสดงคำถามพร้อมตัวเลือก 2. ผู้เล่นกดตัวเลือก A-D 3. SubmitAnswerCommand ตรวจสอบเลือกสุ่มคะแนนจาก ScoringStrategySelector คัดคะแนน บันทึกลง quiz_details 4. แสดงเฉลยและคะแนนที่ได้ 5. ไปข้อต่อไป
Alternative	2a. หมดเวลา → ข้อถัดไป 0 ไม่ส่งคำตอบ / 3a. ตอบข้อเดิมซ้ำหรือรอบจบแล้ว → 409
API	POST /api/v1/quiz-sessions/{id}/answers

UC-06 จบเกมและดูผล	
Actor	ผู้เล่น
Main flow	1. ตอนจบเกมกด "ดูผลคะแนน" 2. ระบบรวม score_earned ทุกข้อเป็น total_score เปลี่ยนสถานะเป็น COMPLETED 3. ส่ง GameCompletedEvent 4. GameCompletionEventListener อัปเดตโปรไฟล์: คะแนนรวม เลเวล (ทุก 100 คะแนน) streak (+1 ถ้าได้คะแนน, 0 ถ้าไม่ได้) 5. แสดงหน้าสรุปผลพร้อมเฉลยทุกข้อ
Alternative	2a. จบช้า → 409
API	POST /api/v1/quiz-sessions/{id}/complete, GET /api/v1/quiz-sessions/{id}/result

UC-07 ดูประวัติการเล่น	
Actor	ผู้เล่น
Main flow	เปิด /history เห็นทุกรอบเรียงจากใหม่ไปเก่า กด "ดูผล" ไปหน้าสรุปผล หรือ "เล่นต่อ" ถ้ายังไม่จบ
API	GET /api/v1/users/{userId}/quiz-sessions

ภาพที่ 2 Use Case Description (UC-01 ถึง UC-07)

Battle IQ - Use Case Description (2/2)

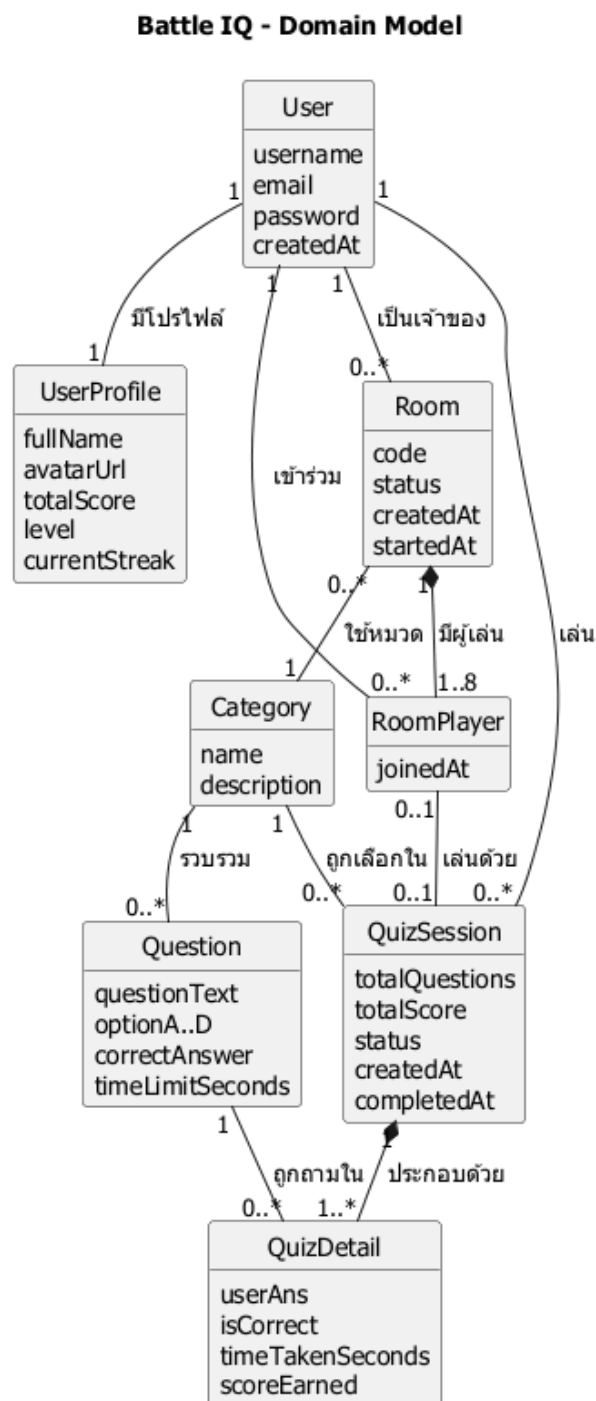
คำอธิบาย use case ประกอบ Use Case Diagram (use-case.png)

UC-08 ตู้อินคัน	
Actor	ผู้เล่น (ไม่ต้อง login)
Main flow	เปิด /leaderboard เห็นผู้เล่นเรียงตามคะแนนรวม แบ่งหน้าละ 10
API	GET /api/v1/leaderboard?page=&size=
UC-09 สร้างห้องแข่ง	
Actor	ผู้เล่น
Main flow	1. เปิด /rooms เลือกหมวดหมู่ กดสร้างห้อง 2. ระบบสร้างรหัส 6 ตัวที่ไม่ซ้ำ บันทึก rooms สถานะ WAITING และเพิ่มผู้สร้างเป็นผู้เล่นคนแรก 3. แสดงหน้าห้องพร้อมรหัสให้ส่งต่อ
Postcondition	ผู้สร้างเป็นเจ้าของห้อง
API	POST /api/v1/rooms
UC-10 เข้าร่วมห้องแข่ง	
Actor	ผู้เล่น
Precondition	มีรหัสห้อง ห้องยังอยู่ในสถานะ WAITING
Main flow	1. ใส่อีเมลห้อง 2. ระบบเพิ่มลง room_players 3. หน้าห้องแสดงรายชื่อผู้เล่น อัปเดตทุก 2 วินาที
Alternative	2a. ห้องเต็มแล้ว → 409 / 2b. ห้องเต็ม (8 คน) → 409 / 2c. อยู่ในห้องแล้ว → คืนข้อมูลห้องเต็ม
API	POST /api/v1/rooms/{code}/join, GET /api/v1/rooms/{code}
UC-11 เริ่มการแข่งขันในห้อง	
Actor	เจ้าของห้อง
Precondition	ห้องมีผู้เล่นอย่างน้อย 2 คน
Main flow	1. เจ้าของห้องกดเริ่มแข่ง 2. ระบบสุ่มคำถาม 1 ชุด แล้วสร้าง quiz_sessions ให้ผู้เล่นทุกคนด้วยคำถามชุดเดียวกัน 3. ห้องเปลี่ยนเป็น IN_PROGRESS 4. ทุกเครื่องถูกพาไปหน้าเล่นเกมของตัวเอง 5. เมื่อทุกคนจบ ห้องเป็น FINISHED แสดงอันดับ
Alternative	1a. ไม่ใช่เจ้าของห้อง → 409 / 1b. ผู้เล่นไม่ถึง 2 → 409
API	POST /api/v1/rooms/{code}/start
UC-12 จัดการหมวดหมู่	
Actor	ผู้ดูแลคำถาม
Main flow	เปิด /admin/categories เพิ่ม แก้ไข ลบหมวดหมู่
Alternative	ชื่อซ้ำ → 409 / ลบหมวดหมู่ที่มีการเล่นแล้ว → 409
API	POST / PUT / DELETE /api/v1/categories
UC-13 จัดการคำถาม	
Actor	ผู้ดูแลคำถาม
Main flow	เปิด /admin/questions ดูคำถามแบบแบ่งหน้า เพิ่ม แก้ไข ลบ
Alternative	เลขไม่ใช่ A-D หรือเวลาไม่อยู่ใน 5-120 วินาที → 400 / ลบคำถามที่เคยถูกใช้ → 409
API	GET / POST / PUT / DELETE /api/v1/questions

ภาพที่ 3 Use Case Description (UC-08 ถึง UC-13)

1.3 Domain Model

แนวคิดหลักของระบบมี 8 แนวคิด ผู้ใช้ (User) มีโปรไฟล์ (UserProfile) หนึ่งเดียว เล่นได้หลายรอบ (QuizSession) แต่ละรอบประกอบด้วยคำถามหลายข้อ (QuizDetail) ซึ่งอ้างถึงคำถาม (Question) ในหมวดหมู่ (Category) ส่วนห้องแข่ง (Room) มีผู้เล่นหลายคนผ่าน RoomPlayer ซึ่งผูกกับรอบของผู้เล่นคนนั้น ดังภาพที่ 4



ภาพที่ 4 Domain Model ของระบบ

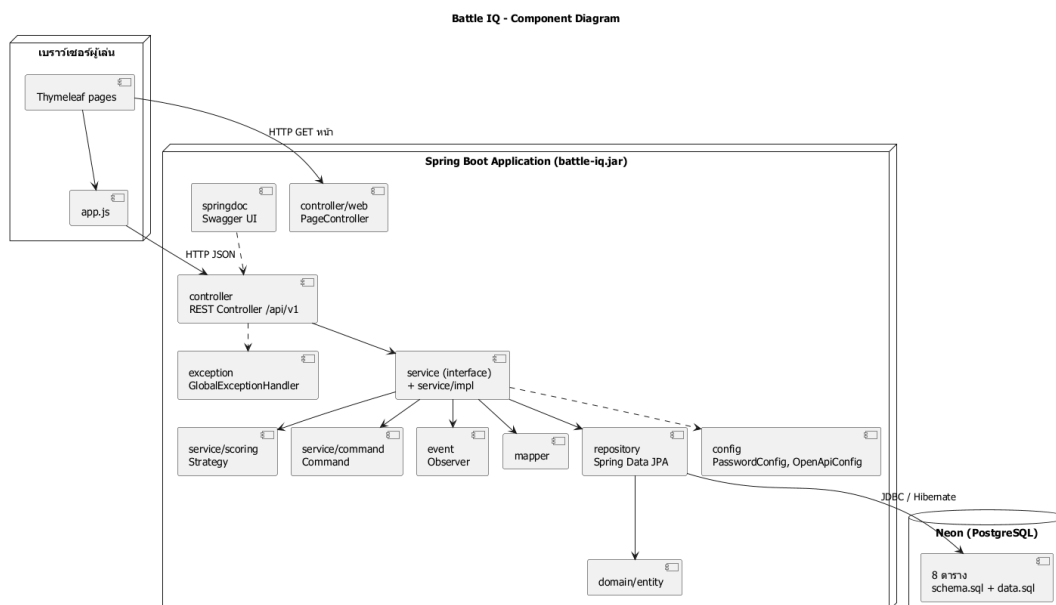
2. การออกแบบระบบ

2.1 สถาปัตยกรรมของระบบ

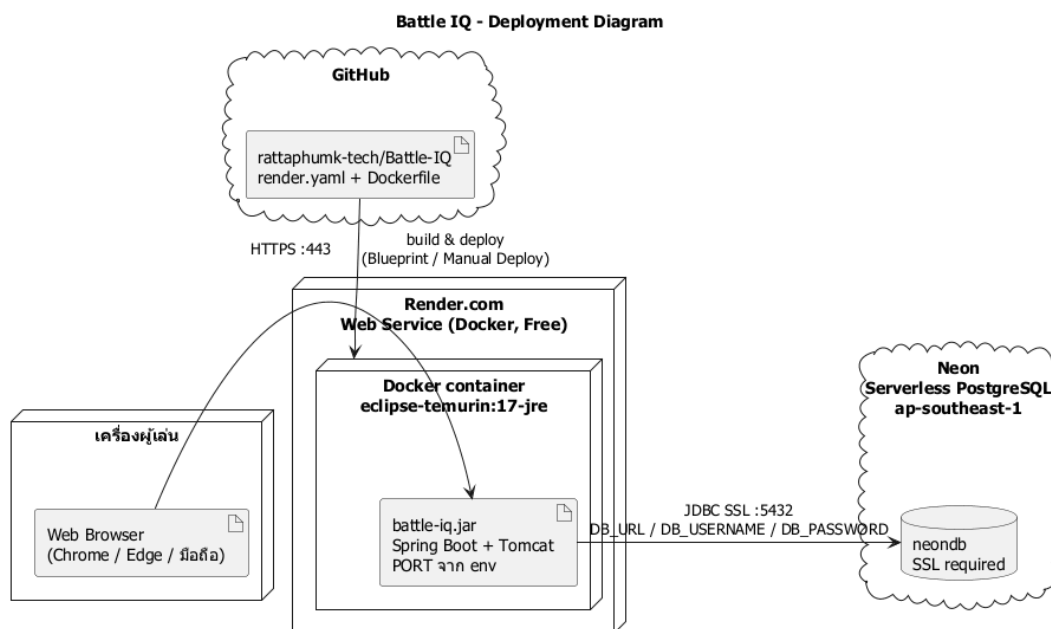
ระบบใช้สถาปัตยกรรมแบบชั้นภายในแอปพลิเคชัน Spring Boot เดียว เบราร์เซอร์โหลดหน้า

Thymeleaf แล้วเรียก REST API ด้วย JavaScript ชั้น Controller ส่งต่อให้ Service ผ่าน interface ชั้น Service

ใช้แบบรูปการออกแบบและ mapper แล้วเรียก Repository ซึ่งเชื่อมกับ PostgreSQL ดังภาพที่ 5 ส่วนการติดตั้งจริงแสดงในภาพที่ 6



ภาพที่ 5 Component Diagram

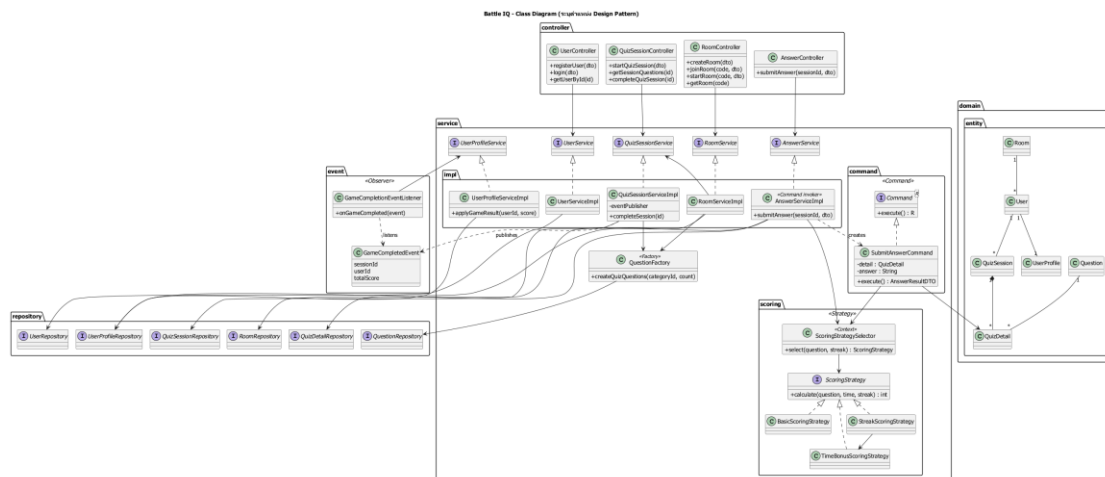


ภาพที่ 6 Deployment Diagram

2.2 การออกแบบคลาส

ภาพที่ 7 แสดงคลาสหลักทุกชั้นพร้อมตำแหน่งของแบบรูปการออกแบบ แพ็กเกจ service.scoring เป็น Strategy แพ็กเกจ service.command เป็น Command แพ็กเกจ event เป็น Observer และ

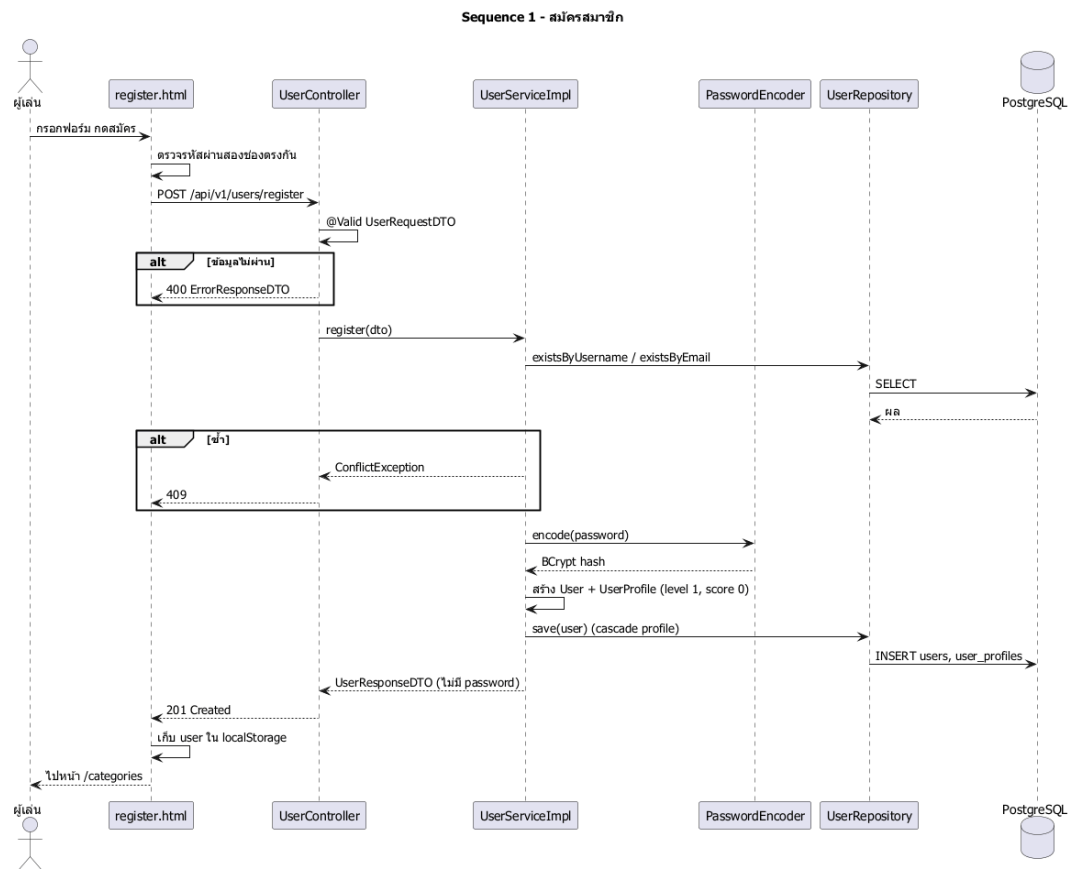
QuestionFactory เป็น Factory ทุก Controller ขึ้นกับ interface ใน service ส่วน implementation อยู่ใน service.impl



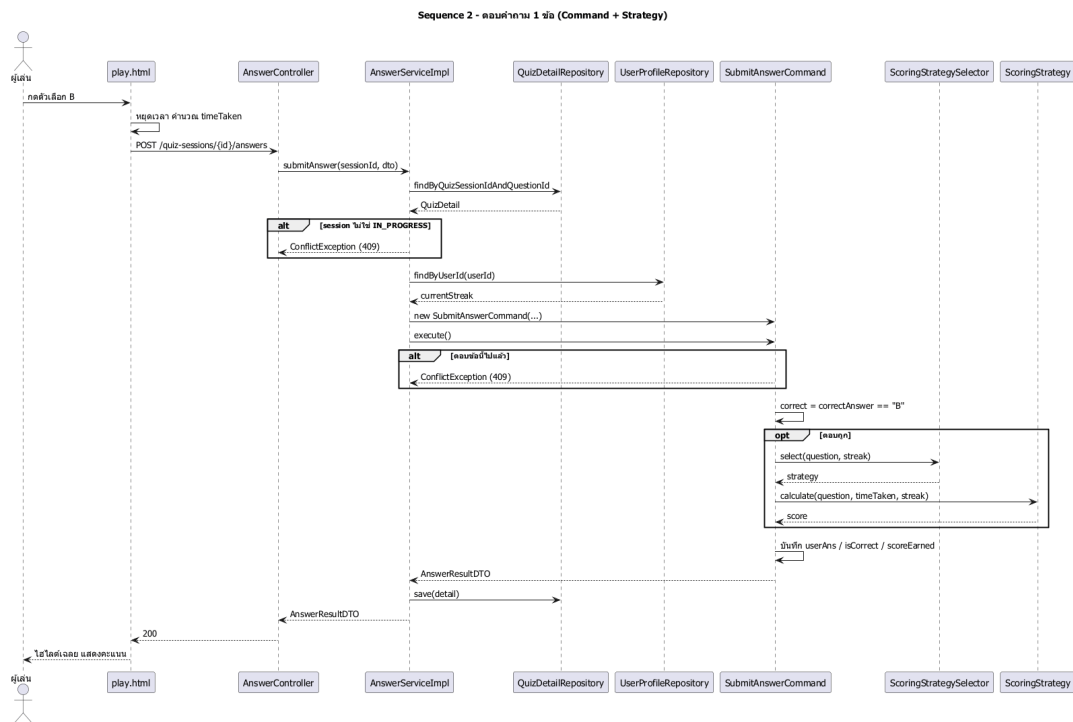
ภาพที่ 7 Class Diagram พร้อมตำแหน่งของแบบรูปการออกแบบ

2.3 แผนภาพแสดงพฤติกรรมเชิงพลวัต

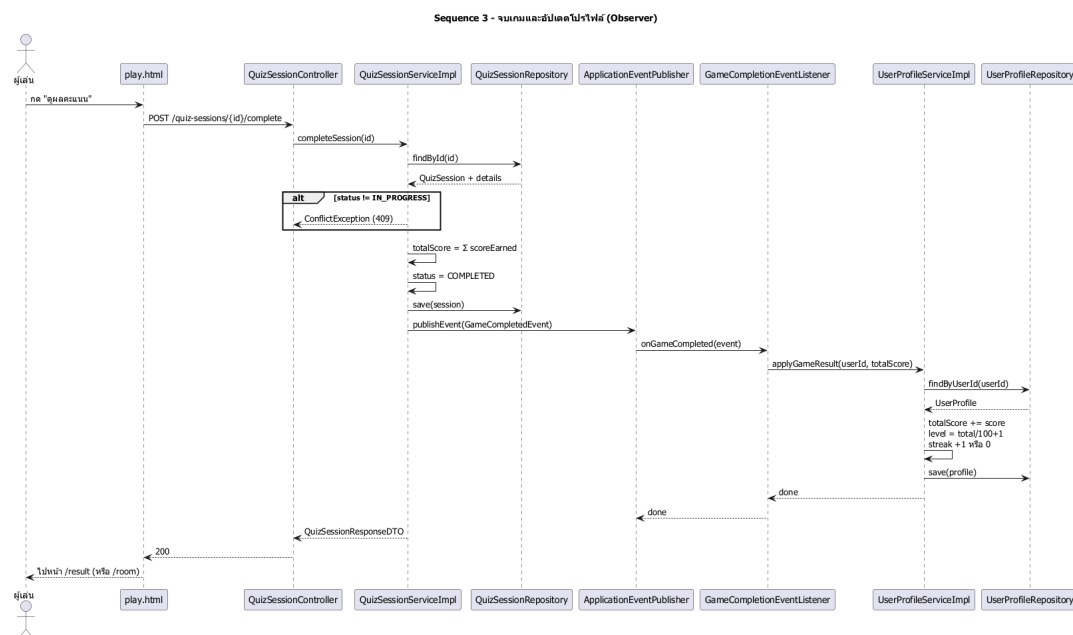
Sequence Diagram 4 สถานการณ์หลักแสดงในภาพที่ 8 ถึง 11 ได้แก่ การสมัครสมาชิก การตอบคำถาม 1 ข้อซึ่งใช้ Command และ Strategy การเล่นเกมซึ่งใช้ Observer และการเริ่มแข่งในห้อง



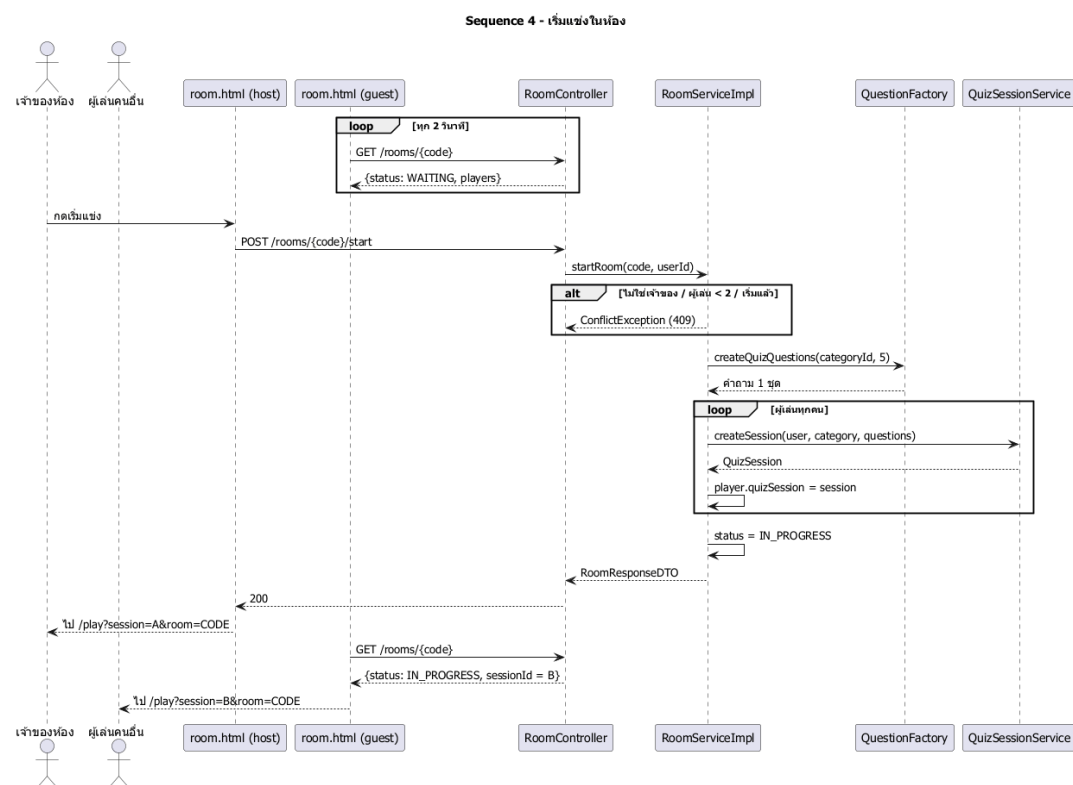
ภาพที่ 8 Sequence Diagram การสมัครสมาชิก



ภาพที่ 9 Sequence Diagram การตอบคำถาม 1 ข้อ (Command และ Strategy)



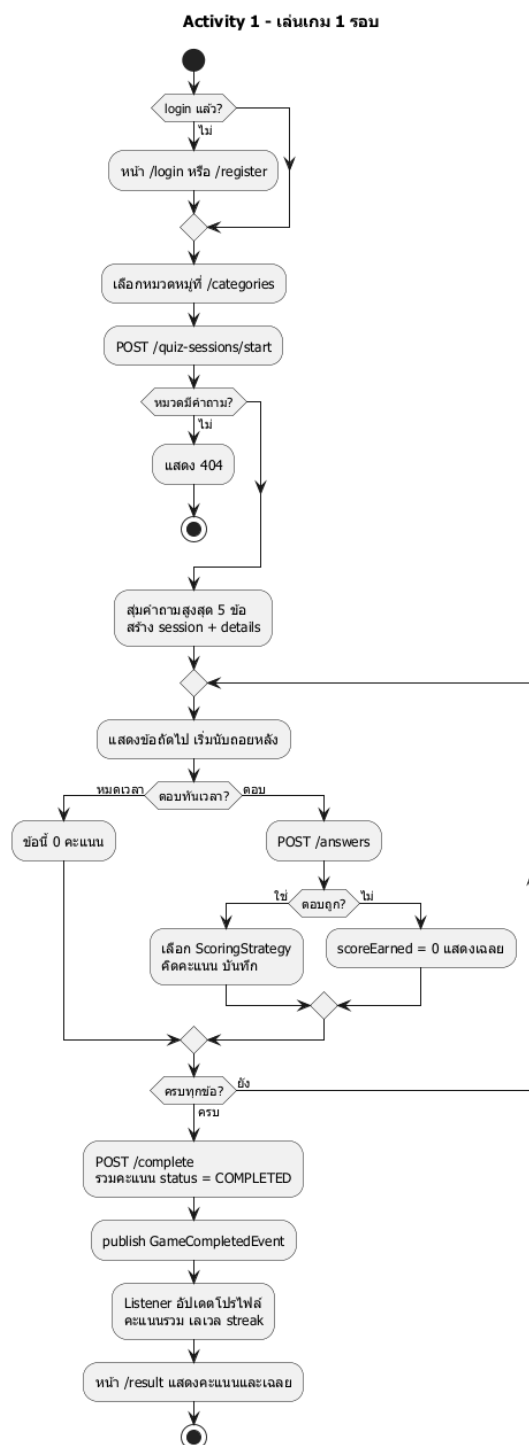
ภาพที่ 10 Sequence Diagram การจบเกมและอัปเดตโปรไฟล์ (Observer)



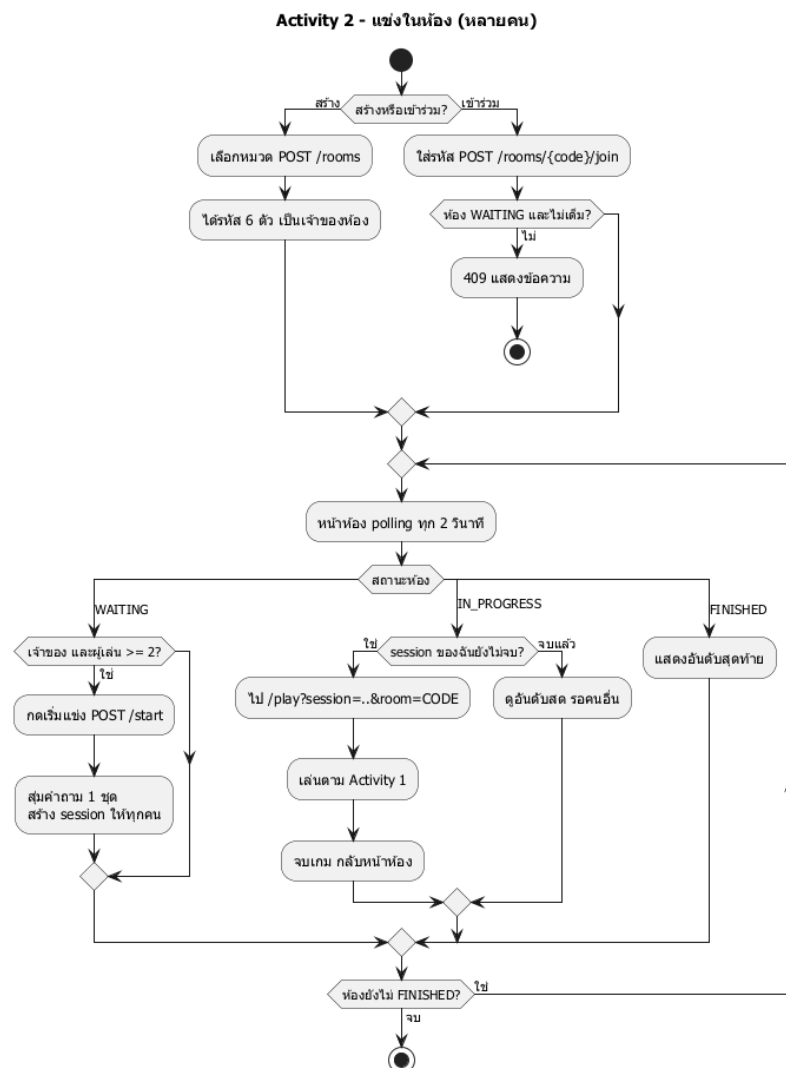
ภาพที่ 11 Sequence Diagram การเริ่มแข่งในห้อง

Activity Diagram แสดงลำดับกิจกรรมของการเล่น 1 รอบและการแข่งในห้องดังภาพที่ 12 และ 13

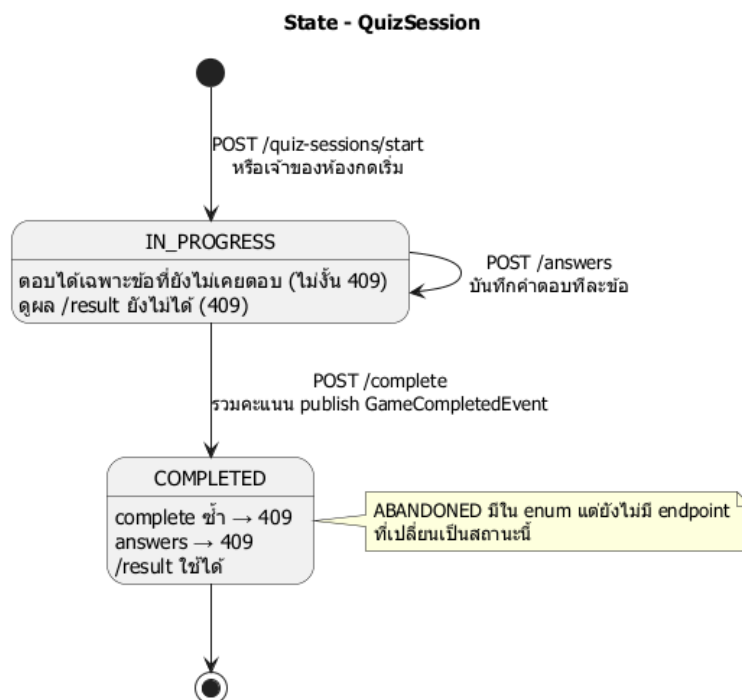
ส่วน State Diagram ของ entity ที่มีสถานะ ได้แก่ QuizSession และ Room แสดงในภาพที่ 14 และ 15



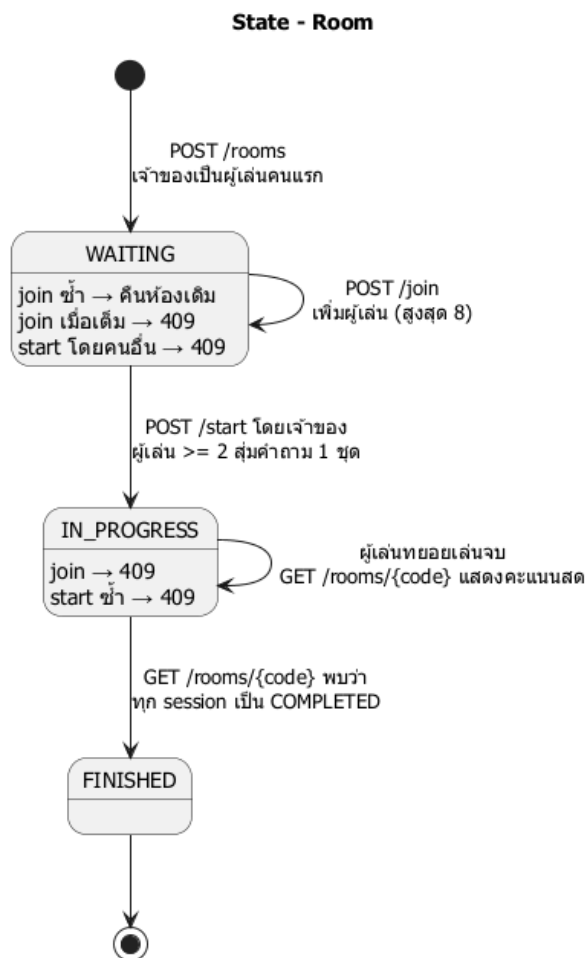
ภาพที่ 12 Activity Diagram การเล่นเกม 1 รอบ



ภาพที่ 13 Activity Diagram การแข่งในห้อง



ภาพที่ 14 State Diagram ของ QuizSession

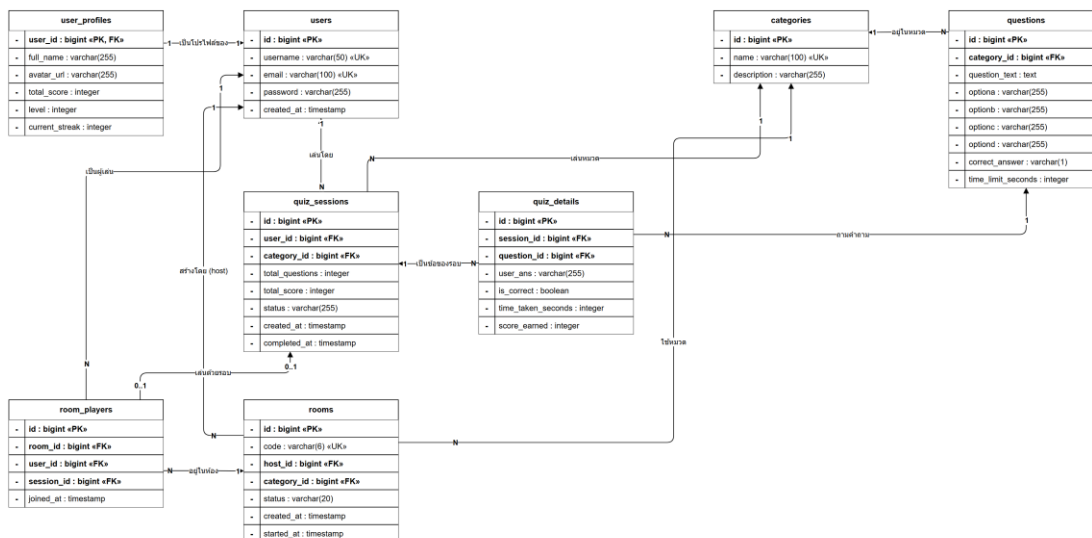


ภาพที่ 15 State Diagram ของ Room

2.4 การออกแบบฐานข้อมูล

คลาสในโดเมนถูกแปลงเป็นตาราง 8 ตาราง ความสัมพันธ์แบบ One-to-One ระหว่าง users กับ user_profiles ใช้ primary key ร่วมกัน (@MapsId) One-to-Many ระหว่าง categories → questions, users → quiz_sessions, quiz_sessions → quiz_details และ rooms → room_players และ Many-to-Many ระหว่าง users กับ rooms ผ่านตารางกลาง room_players ที่มีข้อมูลเพิ่ม (รอบของผู้เล่นในห้อง) ทุกความสัมพันธ์ใช้ Foreign Key และมี index บนคอลัมน์ที่ใช้ค้นหาบ่อย ดังภาพที่ 16 และพจนานุกรมข้อมูลในตารางที่ 8 ถึง 15

Battle IQ - ER Diagram (PostgreSQL, 8 ตาราง)



ภาพที่ 16 ER Diagram ของฐานข้อมูล Battle IQ

ตารางที่ 8 พจนานุกรมข้อมูลตาราง users

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	รหัสผู้ใช้
username	varchar(50)	UK, not null	ชื่อผู้ใช้ a-z 0-9 _ 3-50 ตัว
email	varchar(100)	UK, not null	อีเมล
password	varchar(255)	not null	รหัสผ่าน BCrypt
created_at	timestamp		เวลาสมัคร

ตารางที่ 9 พจนานุกรมข้อมูลตาราง user_profiles

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
user_id	bigint	PK, FK → users	ใช้ PK เดียวกับ users
full_name	varchar(255)		ชื่อที่แสดง
avatar_url	varchar(255)		ลิงก์รูป
total_score	integer		คะแนนสะสม
level	integer		$\text{total_score}/100 + 1$
current_streak	integer		รอบติดกันที่ได้คะแนน

ตารางที่ 10 พจนานุกรมข้อมูลตาราง categories

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	
name	varchar(100)	UK, not null	ชื่อหมวด 2-50 ตัว
description	varchar(255)		คำอธิบาย

ตารางที่ 11 พจนานุกรมข้อมูลตาราง questions

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	
category_id	bigint	FK → categories	หมวด
question_text	text	not null	โจทย์
optiona – optiond	varchar(255)	not null	ตัวเลือก A-D
correct_answer	varchar(1)	not null	A / B / C / D
time_limit_seconds	integer		5-120 ค่าเริ่มต้น 15

ตารางที่ 12 พจนานุกรมข้อมูลตาราง quiz_sessions

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	
user_id	bigint	FK → users	ผู้เล่น
category_id	bigint	FK → categories	หมวดที่เล่น
total_questions	integer		จำนวนข้อ
total_score	integer		คะแนนรวม
status	varchar(255)	not null	IN_PROGRESS / COMPLETED
created_at, completed_at	timestamp		เวลาเริ่ม / จบ

ตารางที่ 13 พจนานุกรมข้อมูลตาราง quiz_details

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	
session_id	bigint	FK → quiz_sessions	รอบ
question_id	bigint	FK → questions	คำถาม

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
user_ans	varchar(255)		คำตอบ A-D
is_correct	boolean		ถูกหรือไม่
time_taken_seconds	integer		เวลาที่ใช้
score_earned	integer		คะแนนข้อนี้

ตารางที่ 14 พจนานุกรมข้อมูลตาราง rooms

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	
code	varchar(6)	UK, not null	รหัสเข้าห้อง
host_id	bigint	FK → users	เจ้าของห้อง
category_id	bigint	FK → categories	หมวดที่แข่ง
status	varchar(20)	not null	WAITING / IN_PROGRESS / FINISHED
created_at, started_at	timestamp		เวลาสร้าง / เริ่ม

ตารางที่ 15 พจนานุกรมข้อมูลตาราง room_players

คอลัมน์	ชนิด	ข้อกำหนด	คำอธิบาย
id	bigint	PK	
room_id	bigint	FK → rooms	ห้อง
user_id	bigint	FK → users	ผู้เล่น (room_id, user_id ไม่ซ้ำ)
session_id	bigint	FK → quiz_sessions	รอบของผู้เล่นในห้อง
joined_at	timestamp		เวลาเข้าห้อง

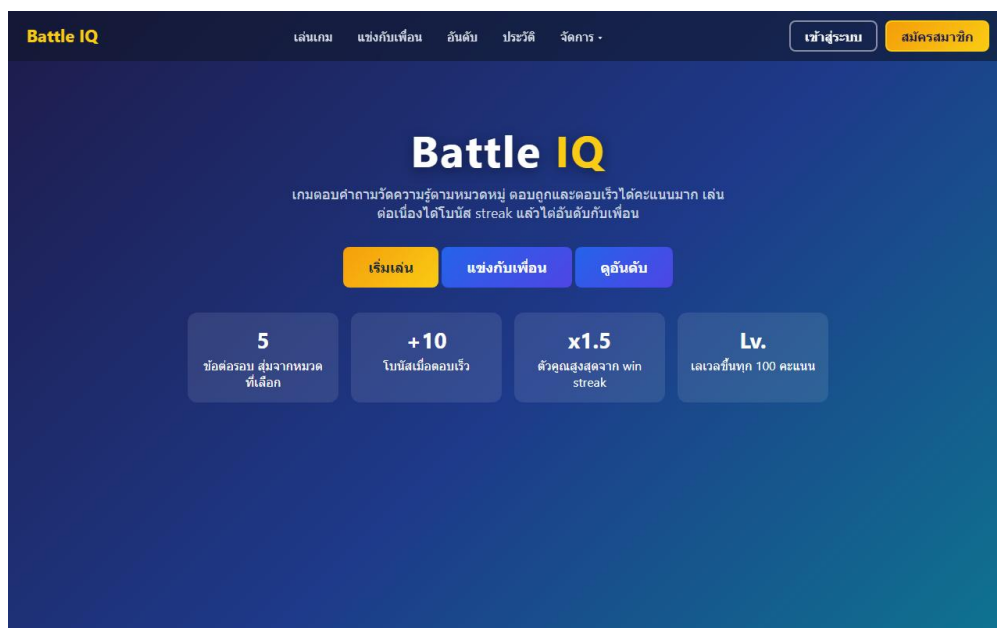
2.5 การออกแบบส่วนติดต่อผู้ใช้

หน้าเว็บทั้ง 13 หน้าใช้โครงร่างเดียวกัน (fragments/layout.html)

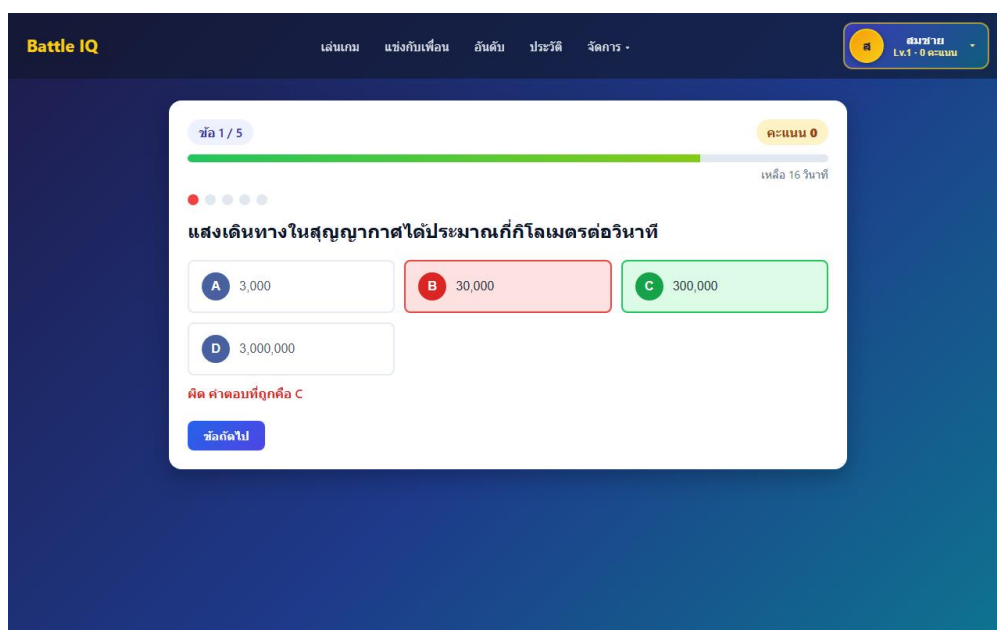
มีเมนูด้านบนและป้ายผู้ใช้ที่แสดงเลเวลกับคะแนนสะสม หน้าหลักที่สำคัญ ได้แก่ หน้าแรก

หน้าเล่นเกมที่มีแถบเวลาและจุดแสดงความคืบหน้า หน้าสรุปผล หน้าห้องแข่งขัน และหน้าอันดับ ดังภาพที่ 17

ถึง 21



ภาพที่ 17 หน้าแรก



ภาพที่ 18 หน้าเล่นเกมหลังตอบคำถาม

Battle IQ เล่นเกม แข่งกับเพื่อน อันดับ ประวัติ จัดการ

สมชาย Lv.1 - 18 คะแนน

สรุปผลการเล่น

หมวดหมู่: วิทยาศาสตร์
ตอบถูก: 1 จาก 5 ข้อ

18 คะแนน

เล่นอีกครั้ง ดูอันดับ

รายละเอียดแต่ละข้อ

คำถาม	ตอบ	เฉลย	คะแนน
พืชใช้แก๊สชนิดใดในการสังเคราะห์ด้วยแสง	A ✓	A	18
ดาวเคราะห์ดวงใดอยู่ใกล้ดวงอาทิตย์ที่สุด	A X	B	0
สูตรเคมีของน้ำคืออะไร	A X	C	0
แสงเดินทางในสุญญากาศได้ประมาณกี่กิโลเมตรต่อวินาที	B X	C	0

ภาพที่ 19 หน้าสรุปผลการเล่น

Battle IQ เล่นเกม แข่งกับเพื่อน อันดับ ประวัติ จัดการ

สมชาย Lv.1 - 18 คะแนน

รอสื่อ
BCB2XM
หมวดหมู่: วิทยาศาสตร์

จบแล้ว
จบการแข่งขัน
กลับ

ผลการแข่งขัน

อันดับ	ผู้เล่น	สถานะ	คะแนน
1	สมชาย 🏆 (คุณ)	เล่นจบแล้ว	18
2	สมหญิง	เล่นจบแล้ว	16

ภาพที่ 20 หน้าห้องแข่งขันเมื่อจบการแข่งขัน

อันดับ	ผู้เล่น	เลเวล	คะแนนรวม
1	Gang	6	503
2	UUClOG77850	3	261
3	จจจจจจ	1	19
4	สมชาย	1	18
5	สมชาย	1	18
6	สมหญิง	1	16
7	สมหญิง	1	16
8	12345	1	0
9	pheerapat	1	0
10	สมชาย	1	0

ภาพที่ 21 หน้าอันดับผู้เล่น

3. การพัฒนาโปรแกรม

3.1 โครงสร้างโปรแกรม

โค้ดอยู่ในแพ็คเกจ `com.battleiq` แบ่งตามชั้น ได้แก่ `controller (REST)` และ `controller.web` (หน้าเว็บ), `service (interface)` กับ `service.impl`, `service.scoring`, `service.command`, `event`, `mapper`, `repository`, `domain.entity`, `domain.enums`, `dto`, `exception` และ `config` ไฟล์ตั้งค่าอ่านค่าฐานข้อมูลจาก `environment variable` ทำให้ย้ายสภาพแวดล้อมได้โดยไม่แก้ไขโค้ด

3.2 ข้อกำหนดของ REST API

ตารางที่ 16 รายการ endpoint ของ REST API

Method	Endpoint	หน้าที่	Status code
POST	/api/v1/users/register	สมัครสมาชิก	201, 400, 409
POST	/api/v1/users/login	เข้าสู่ระบบ	200, 400, 401
GET	/api/v1/users/{id}	ข้อมูลผู้ใช้	200, 404
GET / PUT	/api/v1/profiles/user/{userId}	ดู / แก้โปรไฟล์	200, 400, 404
GET / POST	/api/v1/categories	รายการ / เพิ่มหมวดหมู่	200, 201, 409
GET / PUT / DELETE	/api/v1/categories/{id}	ดู / แก้ / ลบหมวดหมู่	200, 204, 404, 409
GET	/api/v1/questions?page&size&sort	คำถามแบบแบ่งหน้า	200, 400

Method	Endpoint	หน้าที่	Status code
GET	/api/v1/questions/category/{id}	คำถามตามหมวด (ไม่มีเฉลย)	200
POST	/api/v1/questions	เพิ่มคำถาม	201, 400, 404
PUT / DELETE	/api/v1/questions/{id}	แก้ / ลบคำถาม	200, 204, 404, 409
POST	/api/v1/quiz-sessions/start	เริ่มรอบการเล่น	201, 404
GET	/api/v1/quiz-sessions/{id}/questions	คำถามของรอบ	200, 404
POST	/api/v1/quiz-sessions/{id}/answers	ส่งคำตอบ 1 ข้อ	200, 400, 404, 409
POST	/api/v1/quiz-sessions/{id}/complete	จบรอบ รวมคะแนน	200, 404, 409
GET	/api/v1/quiz-sessions/{id}/result	สรุปผลและเฉลย	200, 404, 409
GET	/api/v1/users/{id}/quiz-sessions	ประวัติการเล่น	200, 404
GET	/api/v1/leaderboard?page&size	อันดับผู้เล่น	200
POST	/api/v1/rooms	สร้างห้อง	201, 404
GET	/api/v1/rooms/{code}	สถานะห้องและผู้เล่น	200, 404
POST	/api/v1/rooms/{code}/join	เข้าร่วมห้อง	200, 404, 409
POST	/api/v1/rooms/{code}/start	เริ่มแข่ง	200, 404, 409

ทุกข้อผิดพลาดถูกจัดการที่ GlobalExceptionHandler และตอบในรูปแบบเดียวกัน คือ {"status": 409, "message": "...", "timestamp": "..."} โดย 400 สำหรับข้อมูลไม่ถูกต้อง 401 รหัสผ่านผิด 404 ไม่พบ 409 ขัดแย้งกับสถานะปัจจุบัน และ 415 เมื่อ Content-Type ไม่ใช่ JSON

3.3 การนำแบบรูปการออกแบบไปใช้

3.3.1 Strategy: interface ScoringStrategy มีเมธอดเดียวคือ calculate(question, timeTakenSeconds, currentStreak) มี implementation 3 แบบ ได้แก่ BasicScoringStrategy (10 คะแนน) TimeBonusScoringStrategy (10 + โบนัสตามเวลาที่เหลือสูงสุด 10) และ StreakScoringStrategy (ต่อยอดจาก TimeBonus คุณเพิ่ม 10% ต่อ streak สูงสุด 50%) โดย ScoringStrategySelector เป็นผู้เลือกตามเวลาจำกัดและ streak ของผู้เล่น

```
public interface ScoringStrategy {
    int calculate(Question question, int timeTakenSeconds, int currentStreak);
}
```

```
public ScoringStrategy select(Question question, int currentStreak)
{
    if (question.getTimeLimitSeconds() == null) return
    basicScoringStrategy;
    if (currentStreak >= 2) return streakScoringStrategy;
    return timeBonusScoringStrategy;
}
```

3.3.2 Observer: เมื่อ QuizSessionServiceImpl.completeSession รวมคะแนนเสร็จจะ publish

GameCompletedEvent ผ่าน ApplicationEventPublisher โดยไม่รู้ว่ามีใครรับ

GameCompletionEventListener รับ event ด้วย @EventListener แล้วเรียก

UserProfileService.applyGameResult เพื่ออัปเดตคะแนนสะสม เลเวล และ streak ในทรานแซกชันเดียวกัน

การเพิ่มพฤติกรรมใหม่เมื่อจบเกม เช่น แจกตรารางวัล ทำได้โดยเพิ่ม listener ใหม่โดยไม่แก้ service เดิม

```
eventPublisher.publishEvent(new GameCompletedEvent(session.getId(),
    userId, totalScore));
```

```
@EventListener
public void onGameCompleted(GameCompletedEvent event) {
    userProfileService.applyGameResult(event.getUserId(),
    event.getTotalScore());
}
```

3.3.3 Command: SubmitAnswerCommand ห่อหุ้มการตอบคำถาม 1 ข้อ ได้แก่

ตรวจว่ายังไม่เคยตอบ ตรวจถูกผิด เลือกสูตรและคิดคะแนน แล้วบันทึก QuizDetail โดย

AnswerServiceImpl เป็น invoker ที่โหลดข้อมูล สร้าง command และเรียก execute()

ทำให้ทดสอบตรรกะการตอบได้ด้วยวัตถุธรรมดาโดยไม่ต้องใช้ Spring

```
Command<AnswerResultDTO> command = new SubmitAnswerCommand(
    detail, request.getAnswer(), request.getTimeTakenSeconds(),
    currentStreak, strategySelector);
AnswerResultDTO result = command.execute();
quizDetailRepository.save(detail);
```

3.3.4 Factory: QuestionFactory.createQuizQuestions(categoryId, count)

สุ่มคำถามจากฐานข้อมูลและโยน QuestionNotFoundException เมื่อหมวดไม่มีคำถาม

ใช้ร่วมกันทั้งการเล่นคนเดียวและห้องแข่ง ซึ่งสุ่ม 1 ชุดแล้วสร้างรอบให้ผู้เล่นทุกคนด้วยชุดเดียวกัน

3.4 กติกาคะแนน

ตารางที่ 17 สูตรคิดคะแนนและคะแนนเต็มต่อรอบ

win streak ตอนเริ่มรอบ	สูตรที่ใช้	คะแนนสูงสุดต่อข้อ	คะแนนเต็มต่อรอบ (5 ข้อ)
0 – 1	TimeBonus: $10 + 10 \times \text{เวลาที่เหลือ} \div \text{เวลาทั้งหมด}$	20	100

win streak ตอนเริ่มรอบ	สูตรที่ใช้	คะแนนสูงสุดต่อข้อ	คะแนนเต็มต่อรอบ (5 ข้อ)
2	Streak: TimeBonus $\times$ 1.2	24	120
3	Streak: TimeBonus $\times$ 1.3	26	130
4	Streak: TimeBonus $\times$ 1.4	28	140
5 ขึ้นไป	Streak: TimeBonus $\times$ 1.5	30	150

เลเวลคำนวณจากคะแนนสะสม (ทุก 100 คะแนนขึ้น 1 เลเวล) streak เพิ่ม 1

เมื่อรอบนั้นได้คะแนนมากกว่า 0 และกลับเป็น 0 เมื่อได้ 0 คะแนน

3.5 การติดต่อฐานข้อมูลและการนำขึ้นใช้งาน

ชั้น Repository ใช้ Spring Data JPA โดยประกาศ interface ที่สืบทอด JpaRepository

และเพิ่มเมธอดตามชื่อ เช่น findById, existsByUsername, findByQuizSessionIdAndQuestionId

การสุ่มคำถามใช้ JPQL กับ ORDER BY RANDOM() ร่วมกับ Pageable ตารางถูกสร้างโดย Hibernate และ schema.sql ส่วนข้อมูลตั้งต้น 3 หมวด 15 คำถามอยู่ใน data.sql ซึ่งเขียนให้รันซ้ำได้

การนำขึ้นใช้งานใช้ Dockerfile แบบ multi-stage และ Blueprint ของ Render (render.yaml)

ระบบเข้าถึงได้ที่ <https://battle-iq.onrender.com> โดยค่าเชื่อมต่อนฐานข้อมูลตั้งเป็น environment variable บน Render

4. การทดสอบระบบ

4.1 แผนการทดสอบ

การทดสอบแบ่งเป็น 3 ระดับ ได้แก่ unit test ด้วย JUnit 5 และ Mockito ทดสอบ service, command และ strategy โดยจำลอง repository, integration test ด้วย Spring Boot Test และ MockMvc ยิง HTTP ผ่าน controller จริงและฐานข้อมูลจริงโดย rollback ทุกกรณี และ system test ด้วยมือผ่านหน้าเว็บตาม checklist กรณีทดสอบออกแบบด้วย equivalence partitioning, boundary value analysis, negative testing และ state-based testing รายละเอียดทุกกรณีอยู่ใน test/README.md ของ repository

ตารางที่ 18 สรุปกรณีทดสอบอัตโนมัติแยกตามคลาส

คลาสทดสอบ	ระดับ	สิ่งที่ทดสอบ	จำนวน	ผล
UserServiceTest	Unit	สมัคร (สร้างโปรไฟล์ เข้ารหัส) เข้า login ถูก/ผิด	8	ผ่าน
UserProfileServiceTest	Unit	คะแนนรวม เลเวล streak แก่โปรไฟล์	4	ผ่าน

คลาสทดสอบ	ระดับ	สิ่งที่ทดสอบ	จำนวน	ผล
ScoringStrategyTest	Unit	สูตร 3 แบบ ค่าขอบ เพดาน streak	4	ผ่าน
SubmitAnswerCommandTest	Unit	ตอบถูก ตอบผิด ตอบซ้ำ	3	ผ่าน
QuizSessionServiceTest	Unit	รวมคะแนน publish event จบซ้ำ	3	ผ่าน
CategoryServiceTest	Unit	เพิ่ม ชื่อซ้ำ ไม่พบ	3	ผ่าน
RoomServiceTest	Unit	สร้าง เข้าร่วม เริ่ม คำถามชุดเดียวกัน จบห้อง	7	ผ่าน
UserApiIntegrationTest	Integration	status code 201/400/401/404/409 ค่าขอบรหัสผ่าน	9	ผ่าน
ApiValidationIntegrationTest	Integration	ข้อมูลยาวเกิน ฟิลด์รูปแบบ sort ฟิลด์ content-type ฟิลด์	12	ผ่าน
BattleIqApplicationTests	Smoke	แอปสตาร์ทและต่อฐานข้อมูล	1	ผ่าน

ตารางที่ 19 ตัวอย่างกรณีทดสอบแบบค่าขอบและกรณีผิดปกติของ API

รหัส	ข้อมูลเข้า	ผลที่คาด	ผล
TC-A02	รหัสผ่าน 5 ตัวอักษร	400	ผ่าน
TC-A03	รหัสผ่าน 6 ตัวอักษร	201	ผ่าน
TC-V03	รหัสผ่าน 73 ตัวอักษร (เกิน BCrypt)	400 (ก่อนแก้ไขได้ 500)	ผ่าน
TC-V02	ชื่อที่แสดง 500 ตัวอักษร	400 (ก่อนแก้ไขได้ 500)	ผ่าน
TC-V08	sort ด้วยชื่อ field ที่ไม่มี	400 (ก่อนแก้ไขได้ 500)	ผ่าน
TC-V11	Content-Type เป็น text/plain	415 ในรูปแบบมาตรฐาน	ผ่าน
TC-C03	ตอบคำถามข้อเดิมซ้ำ	ConflictException (409)	ผ่าน
TC-R05	เริ่มห้องที่มีผู้เล่นคนเดียว	ConflictException (409)	ผ่าน

4.2 ผลการทดสอบ

การทดสอบอัตโนมัติครั้งล่าสุด (8 ตุลาคม 2569) รวม 54 กรณี ผ่านทั้งหมด รายงานจาก Maven Surefire เก็บไว้ที่ test/report/ การทดสอบระบบด้วยมือ 9 รายการ ได้แก่ สมัคร เลื่อนหมวด เล่นจนจบ ดูผล โปรไฟล์ ประวัติและอันดับ จัดการข้อมูล จอแคบ และห้องแข่งขันด้วย 2 เบราวเซอร์ ผ่านทั้งหมดบนระบบที่ติดตั้งจริง การทดสอบด้วยข้อมูลผิดปกติพบข้อบกพร่อง 4 จุดที่ทำให้ API ตอบ 500 ซึ่งแก้ไขโดยเพิ่มการตรวจสอบความยาวข้อมูลเข้าและการจัดการ exception เพิ่มเติม

บทที่ 5 บทสรุป

1. สรุปผลการดำเนินโครงการ

คณะผู้จัดทำพัฒนาเว็บแอปพลิเคชัน Battle IQ ได้ครบตามขอบเขตที่กำหนด ระบบรองรับการสมัครสมาชิก การเล่นเกมตอบคำถามแบบจับเวลา การคิดคะแนนที่ขึ้นกับความเร็วและ win streak เลเวล อันดับ ประวัติการเล่น การจัดการคำถาม และโหมดห้องแข่งขันหลายคน มี REST API 22 endpoint พร้อม Swagger ฐานข้อมูล 8 ตารางที่มีความสัมพันธ์ครบทั้ง 3 แบบ ชุดทดสอบอัตโนมัติ 54 กรณี และนำขึ้นใช้งานจริงที่ <https://battle-iq.onrender.com>

ด้านการออกแบบ ระบบเป็นไปตามสถาปัตยกรรมแบบชั้นอย่างเคร่งครัด Controller ขึ้นกับ interface ของ Service และการแปลงข้อมูลแยกไปที่ mapper แบบรูปการออกแบบทั้งสามแก้ปัญหาที่เกิดขึ้นจริง ได้แก่ Strategy ทำให้เพิ่มสูตรคะแนนได้โดยไม่ต้องแก้โค้ดตอบคำถาม Observer ทำให้ส่วนเกมไม่ต้องรู้จักส่วนโปรไฟล์ และ Command ทำให้ทดสอบตรรกะการตอบได้โดยไม่ต้องใช้ Spring การแบ่งงานตามส่วนของระบบทำให้สมาชิกทั้ง 5 คนพัฒนาในกิจของตนและรวมงานผ่าน pull request ได้โดยแทบไม่เกิดความขัดแย้งของไฟล์

2. ข้อจำกัดของระบบ

- 2.1 ไม่มีระบบยืนยันตัวตนฝั่งเซิร์ฟเวอร์ สถานะการเข้าสู่ระบบเก็บในเบราว์เซอร์ และ API ทุกเส้นเรียกได้โดยไม่ต้องยืนยันตัวตน จึงไม่เหมาะกับการใช้งานที่ต้องการความปลอดภัยสูง
- 2.2 ไม่แยกสิทธิ์ผู้ดูแลระบบ ผู้ใช้ทุกคนเข้าหน้าจัดการคำถามได้ และ API ดึงคำถามแบบแบ่งหน้าส่งเฉลยออกไปด้วย
- 2.3 ห้องแข่งขันใช้การ polling ทุก 2 วินาที จึงมีความหน่วงและไม่ใช้เรียลไทม์แท้จริง และห้องที่มีผู้เล่นไม่เล่นจนจบจะค้างในสถานะกำลังแข่ง
- 2.4 เวลาต่อข้อนับที่เครื่องผู้เล่น ผู้เล่นที่แก้ไขสคริปต์ในเบราว์เซอร์อาจส่งเวลาที่ผิดจริงได้
- 2.5 บริการ Render แพลนฟรีจะหยุดทำงานเมื่อไม่มีการใช้งาน การเปิดครั้งแรกจึงใช้เวลาาวหนึ่งนาที

3. ปัญหาอุปสรรคและแนวทางแก้ไข

ตารางที่ 20 ปัญหาที่พบระหว่างดำเนินงานและแนวทางแก้ไข

ปัญหา	สาเหตุ	แนวทางแก้ไข
กึ่ง develop คอมไพล์ไม่ผ่านหลังจัดโครงสร้างโฟลเดอร์ใหม่	คลาส main ถูกลบและมีไฟล์ซ้ำสองชุด ภายหลังมีไฟล์ทดสอบถูกวางใน src/main	กู้ไฟล์จากประวัติ git ลบชุดซ้ำ ย้ายเทสไป src/test และกำหนดให้รันเทสก่อนเปิด pull request
ทีมไม่มีฐานข้อมูลร่วมกันและเครื่องสมาชิกไม่มี MySQL	ตั้งค่าฐานข้อมูลไว้เฉพาะในเครื่อง	ย้ายไป PostgreSQL บน Neon ใช้ร่วมกันทั้งทีม อ่านค่าเชื่อมต่อจากไฟล์ .env ที่ไม่ commit
การสุ่มคำถามใช้ฟังก์ชัน RAND() ของ MySQL	JPQL เรียกฟังก์ชันเฉพาะฐานข้อมูล	เปลี่ยนเป็น RANDOM() ของ PostgreSQL และทดสอบกับฐานข้อมูลจริง
API ตอบ 500 เมื่อรับข้อมูลยาวผิดปกติ	DTO ไม่ได้จำกัดความยาวให้ตรงกับคอลัมน์ และ BCrypt รับได้ไม่เกิน 72 ไบต์	เพิ่ม @Size ให้ทุก field และจัดการ exception ของ sort และ content-type ให้ตอบ 400/415
ปุ่มที่ควรซ่อนยังแสดงบนหน้าเว็บ	CSS กำหนด display ของปุ่มทับ attribute hidden ของเบราวเซอร์	เพิ่มกฎ [hidden] { display: none !important }
Render ไม่ deploy อัตโนมัติเมื่อ push	สมาชิกที่เชื่อม Render ไม่ใช่เจ้าของ repository จึงติดตั้ง webhook ไม่ได้	ใช้ Manual Deploy ชั่วคราว และให้เจ้าของ repository เชื่อม GitHub กับ Render

4. ข้อเสนอแนะในการพัฒนาต่อไป

- 4.1 เพิ่มการยืนยันตัวตนด้วย JWT หรือ session ผังเซิร์ฟเวอร์ และแยกบทบาทผู้ดูแลระบบด้วย Spring Security
- 4.2 เปลี่ยนห้องแข่งขันเป็น WebSocket เพื่อให้เริ่มพร้อมกันและเห็นคะแนนทันที และนับเวลาต่อข้อผังเซิร์ฟเวอร์
- 4.3 เพิ่มระบบตรารางวัล (badge) โดยเพิ่ม listener ของ GameCompletedEvent และสูตรคะแนนตามระดับความยากของคำถามโดยเพิ่ม ScoringStrategy ใหม่ ซึ่งโครงสร้างปัจจุบันรองรับโดยไม่ต้องแก้ไขโค้ดเดิม
- 4.4 จัดการสถานะ ABANDONED สำหรับรอบและห้องที่ถูกทิ้ง และเพิ่มงานเบื้องหลังปิดห้องที่ค้าง
- 4.5 เพิ่ม GitHub Actions ให้รันเทสทุก pull request และ deploy อัตโนมัติ

เอกสารอ้างอิง

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.
2. Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall.
3. VMware. (2026). Spring Boot Reference Documentation. สืบค้นจาก <https://docs.spring.io/spring-boot/>
4. VMware. (2026). Spring Framework Documentation: Application Events. สืบค้นจาก <https://docs.spring.io/spring-framework/reference/core/beans/context-introduction.html>
5. The PostgreSQL Global Development Group. (2026). PostgreSQL Documentation. สืบค้นจาก <https://www.postgresql.org/docs/>
6. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.
7. Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference.
8. Render. (2026). Render Documentation: Blueprint Specification. สืบค้นจาก <https://render.com/docs/blueprint-spec>
9. Thymeleaf. (2026). Tutorial: Using Thymeleaf. สืบค้นจาก <https://www.thymeleaf.org/documentation.html>

ภาคผนวก

ภาคผนวก ก คู่มือการติดตั้งและใช้งาน

1. การติดตั้งเพื่อพัฒนา

- 1.1 ติดตั้ง Java 17 แล้ว clone repository <https://github.com/rattaphumk-tech/Battle-IQ>
- 1.2 สร้างไฟล์ code/battle-iq/.env จาก .env.example ใส่ DB_URL (jdbc:postgresql://...), DB_USERNAME, DB_PASSWORD ของ PostgreSQL
- 1.3 รัน .mvnw.cmd spring-boot:run ในโฟลเดอร์ code/battle-iq แล้วเปิด <http://localhost:8080> ตารางและข้อมูลตั้งต้นจะถูกสร้างอัตโนมัติ
- 1.4 รันทดสอบด้วย .mvnw.cmd test และสร้างรายงานด้วย .mvnw.cmd surefire-report:report-only
- 1.5 รันด้วย Docker จากรากของ repository ด้วย docker compose up --build

2. การใช้งาน

- 2.1 สมัครสมาชิกที่ /register แล้วเลือกหมวดหมู่ที่ /categories เพื่อเริ่มเล่น
- 2.2 ตอบคำถาม 5 ข้อภายในเวลาที่กำหนด ดูเฉลยหลังตอบแต่ละข้อ และดูสรุปผลเมื่อจบ
- 2.3 แข่งกับเพื่อนที่ /rooms สร้างห้องแล้วส่งรหัส 6 ตัวให้เพื่อน
เจ้าของห้องกดเริ่มเมื่อมีผู้เล่นอย่างน้อย 2 คน
- 2.4 ดูอันดับที่ /leaderboard ประวัติที่ /history และจัดการคำถามที่ /admin/questions
- 2.5 เอกสาร API ที่ /swagger-ui.html

ภาคผนวก ข สถิติการทำงานร่วมกันบน GitHub

ข้อมูลจาก git log ของ repository ณ วันที่ 8 ตุลาคม 2569 นับเฉพาะ commit ที่ไม่ใช่ merge

ตารางที่ 21 จำนวน commit และ pull request ของสมาชิก

สมาชิก	บัญชี GitHub	จำนวน commit	ส่วนงานหลัก
นายพีรพัฒน์ ป้องกันยา	PheerapatCSKKU	56	ผู้ใช้ โปรไฟล์ ห้องแข่ง ฐานข้อมูล เทส เอกสาร
นายรัฐภูมิ เกิดพระจัน	BCT15, rattaphumk-tech	26	entity ระบบเล่นเกม exception handler Docker
นายณพวิทย์ วงษ์ประเสริฐ	MoNa1chz	16	controller หมวดหมู่ leaderboard deploy
นายธนกฤต ละครพล	Dream	15	repository คะแนน ผลการเล่น schema
นายก่อภพ โชควิริยะ	Shade010	13	DTO คำถาม ส่งคำตอบ Swagger หน้าเว็บ

การรวมงานทั้งหมดทำผ่าน pull request 23 รายการเข้าถึง develop โดยมีสมาชิกอีกคนเป็นผู้ review

ประวัติผู้เขียน

นายรัฐภูมิ เกิดพระจีน

รหัสนักศึกษา 673380057-5 นักศึกษาชั้นปีที่ 2 สาขาวิชาวิทยาการคอมพิวเตอร์
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น
หน้าที่ในโครงการ: ระบบเล่นเกม: Entity, QuizSessionService, QuestionFactory,
GlobalExceptionHandler, หน้าเล่นเกม, Docker

นายพีรพัฒน์ ป้องกันยา

รหัสนักศึกษา 673380053-3 นักศึกษาชั้นปีที่ 2 สาขาวิชาวิทยาการคอมพิวเตอร์
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น
หน้าที่ในโครงการ: ผู้ใช้และโปรไฟล์ (Observer), ห้องแข่งขัน, ฐานข้อมูลและ deploy, แผนทดสอบ,
เอกสาร

นายก้องภพ โชควิริยะ

รหัสนักศึกษา 673380030-5 นักศึกษาชั้นปีที่ 2 สาขาวิชาวิทยาการคอมพิวเตอร์
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น
หน้าที่ในโครงการ: คำถามและการส่งคำตอบ (Command), Swagger, โครงหน้าเว็บ

นายธนกฤต ละครพล

รหัสนักศึกษา 673380269-0 นักศึกษาชั้นปีที่ 2 สาขาวิชาวิทยาการคอมพิวเตอร์
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น
หน้าที่ในโครงการ: การคิดคะแนน (Strategy), ผลการเล่นและประวัติ, Repository, schema.sql

นายณพวิทย์ วงษ์ประเสริฐ

รหัสนักศึกษา 673380266-6 นักศึกษาชั้นปีที่ 2 สาขาวิชาวิทยาการคอมพิวเตอร์
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น
หน้าที่ในโครงการ: หมวดหมู่และ Leaderboard, Controller ชุดแรก, Deploy บน Render